Systems Analysis and Design (CO3115)

Software Requirements Specification

for

RESTAURANT MANAGEMENT SYSTEM

Version 1.0 approved

Prepared by

Trần Phạm Đinh Hương	2352455
Văn Công Khoa	2352594
Võ Trúc Sơn	2252720
Võ Việt Huy	2352417
Vũ Hoàng Long	2352707

Ho Chi Minh City University of Technology (HCMUT) – VNU-HCM

April 2026

Table of contents

1. Introduction.....5
1.1 Purpose.....5
1.2 Document Conventions.....5
1.3 Intended Audience and Reading Suggestions.....5
1.4 Project Scope.....6
1.5 References.....7
2. Overall Description.....8
2.1 Product Perspective.....8
2.2 Product Features.....8
2.2.1 Access Control and Administration.....8
2.2.2 Menu and Inventory Management.....9
2.2.3 Order Processing and Service.....9
2.2.4 Kitchen Operations.....9
2.3 User Classes and Characteristics.....10
2.3.1 Waiter.....10
2.3.2 Chef.....11
2.3.3 Admin.....12
2.4 Operating Environment.....13
2.5 Design and Implementation Constraints.....14
2.6 User Documentation.....15
2.7 Assumptions and Dependencies.....16
3. Data Flow Diagrams.....17
3.1 Context Diagram.....17
3.2 Diagram 0.....18
3.3 Child Diagrams.....20
3.3.1 Process 1: Process Customer Service / Order.....20
3.3.2 Process 2: Manage Menu & Inventory.....21
3.3.3 Process 3: Execute Kitchen Operation.....22
3.3.4 Process 4: Authenticate User.....23
3.3.5 Process 5: Manage User Account.....24
4. System Features.....25
4.1 System Feature 1: Waiter Order Processing and Table Session Management.....25
4.2 System Feature 2: Live Order Modification.....26
4.3 System Feature 3: Session-Based Billing.....27
4.4 System Feature 4: Chef Menu, Inventory and Kitchen Operations.....28
4.5 System Feature 5: System Administration.....30
5. Nonfunctional Requirements.....32
5.1 Performance Requirements.....32
5.2 Availability and Reliability Requirements.....32
5.3 Safety Requirements.....32
5.4 Security Requirements.....33
5.5 Usability Requirements.....33
5.6 Maintainability and Portability Requirements.....33

5.7 Interoperability and Compliance Requirements....33
6. Other Requirements....34
6.1 Database Requirements....34
6.2 Deployment and Installation Requirements....34
6.3 Reuse Objective Requirements....34
6.4 Documentation Requirements....34
7. System Analysis....35
7.1 Use case Diagrams....35
7.2 Use case Scenarios....36
7.3 Activity Diagrams....47
7.3.1 Waiter Order Processing....47
7.3.2 Daily Kitchen Workflow....48
7.4 Sequence Diagrams....49
7.4.1 Waiter Order Processing....49
7.4.2 Daily Kitchen Workflow....50
7.4.3 System Administration....51
7.5 Class Diagrams....52
8. System Design....54
8.1 Architectural Design....54
8.1.1 Architecture and Design Pattern....54
8.1.2 Architectural Justification and Comparison....54
8.1.3 Logical View....55
8.1.4 Physical View....56
8.2 Database Design....57
8.3 UI Design....61
8.3.1 Authentication Interface....61
8.3.2 Waiter Interface (Order Processing & Table Management)....62
8.3.3 Chef Interface (Kitchen Operations)....64
8.3.4 Admin Interface (System Administration & Reporting)....65
9. Conclusions....67

Revision History

Name	Date	Reason For Changes	Version
System Overview	8/02/2026	Modification for Assignment 1	1.1
System Analysis	15/03/2026	Modification for Assignment 2	1.2
System Design	12/4/2026	Modification for Assignment 3	1.3

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the software requirements for a Restaurant Order Management System designed for a single restaurant without branches. The system focuses on supporting the food ordering process, in which customers place orders through a tablet provided by restaurant staff and orders are managed within the system.

The document represents the initial of the system specification, the scope of which is limited to the order management subsystem of the overall restaurant management system. It covers functionalities related to menu presentation, order creation, order modification, and order aggregation by table. Other subsystems, such as payment processing and detailed kitchen workflow management, are not described in this SRS.

The purpose of this document is to clearly specify the requirements of the order management subsystem and to provide a common reference for stakeholders involved in system analysis and design.

1.2 Document Conventions

The Software Requirements Specification (SRS) document follows a standard structure and presentation style for software requirements documentation.
The following conventions are used throughout this document:

- Standard section and subsection numbering (e.g., 1.1, 2.3) is applied to organize the content and improve readability.
- Headings are used to clearly indicate the purpose and scope of each section.
- No special fonts, color coding, or typographical highlighting are used beyond standard formatting.
- All requirement descriptions are written in clear and formal English to minimize ambiguity.
- Requirement priorities are not explicitly defined in this document. All requirements are considered to have equal priority unless otherwise stated.

1.3 Intended Audience and Reading Suggestions

Intended Audience

The Software Requirements Specification (SRS) document is intended for stakeholders involved in the evaluation, analysis, design, and development of the Restaurant Order Management System. The

intended readers include lecturers and reviewers, project development team members (such as system analysts, designers, and developers), testers and evaluators, and documentation writers.

Document Organization

This SRS is structured to describe the system at progressively increasing levels of detail. The document is organized as follows:

- Section 1 introduces the purpose, scope, document conventions, intended audience, and overall organization of the SRS.
- Section 2 provides a high-level description of the system, including its context, major features, user classes, operating environment, design and implementation constraints, assumptions, and dependencies.
- Section 3 presents data flow diagrams that illustrate the functional decomposition of the system and the flow of data between system processes.
- Section 4 specifies the detailed functional requirements of the system, organized by major system features.
- Section 5 defines the nonfunctional requirements, including performance, security, and other software quality attributes.
- Section 6 captures additional system requirements that are not covered in other sections.
- Section 7 presents system analysis models, including use case diagrams, activity diagrams, sequence diagrams, and class diagrams.
- Section 8 provides conclusions of the main contents of this document.

Recommended Reading Sequence

Readers are encouraged to begin with Section 1 to understand the purpose, scope, and structure of the document, followed by Section 2 for an overview of the system context and characteristics. Lecturers and reviewers may focus primarily on Sections 1 through 7 to assess requirement completeness and consistency, while project development team members, testers, and documentation writers are recommended to read the document sequentially, as later sections build upon definitions and assumptions introduced earlier.

1.4 Project Scope

The software being specified is a Tablet-based Restaurant Management System designed for a single-unit restaurant. Its primary purpose is to digitize the food ordering workflow and centralize system management.

Objectives of the project:

+ **Staff-Assisted Digital Ordering:** Waiters log into the system on tablets and provide them to customers to browse a logically organized menu and place orders.

+ **Dynamic Inventory Control:** The system tracks the specific number of portions for each dish as defined by the Chef. Once a dish reaches zero portions, the system automatically prevents further orders.

+ **Customized Orders:** Customers can add specific notes to individual dishes during the ordering process.

+ **Table Session Management:** The system manages billing based on table occupancy, tracking all orders from the moment a table is occupied until it becomes vacant.

+ **Chef's Management Console:** Allows Chefs to reset portion counts daily, adjust pricing, and add or remove dishes. It includes a "Menu Copy" feature to duplicate the previous day's menu.

+ **System Administration:** Provides an interface for the Admin to manage user accounts, including adding, deleting, and modifying credentials for both Waiters and Chefs.

User Roles: The scope includes three primary roles: **Waiter** (order processing), **Chef** (inventory/menu management), and **Admin** (account and system management).

Benefits: This system improves order accuracy, prevents the selling of out-of-stock items, and ensures secure access through role-based account management.

1.5 References

Restaurant Management System Requirements Brief: Created by the Course Instructor/Client, version 1.0, January 2026. This document contains the original project description, including the single-unit restaurant scope and core ordering requirements.

Professional Skills for Engineers - Assignment 1 Template: Provided by the University Department, version 2.1, dated January 2026. This template defines the mandatory sections and formatting conventions for the project report.

Restaurant System UI/UX Style Guide: Developed by the Dev Team, version 1.0 (Draft), dated February 4, 2026. Located in the Internal Project Repository, this guide specifies the interface design for the tablet-based ordering system.

Software Architecture Document (SAD) for Web Applications: Authored by the Dev Team, version 1.1, dated February 2026. Available in the Internal Technical Documentation folder, providing details on the system's web-based deployment and server synchronization.

2. Overall Description

2.1 Product Perspective

Currently, many restaurants operate using a traditional manual workflow where waiters record customer orders on paper notes. This method is often inefficient and prone to human error. Common issues include verbal miscommunication in noisy environments or illegible handwriting, which can lead to incorrect orders being prepared. Additionally, managing billing is particularly challenging when customers place multiple separate orders during a meal. Waiters must manually track and merge several loose paper tickets for a single table, which can lead to calculation errors or missed items on the final invoice. Furthermore, the physical delay in transferring tickets to the kitchen slows down service, and manual tracking means staff are often unaware when items run out of stock. These limitations frequently result in service delays, billing discrepancies, and an inconsistent customer experience.

To resolve these challenges, the Restaurant Management System (RMS) replaces manual ordering with a mobile web application that automates the entire service lifecycle. The system ensures order accuracy by replacing handwriting with digital selection, effectively eliminating errors caused by illegible notes or mishearing. By transmitting orders immediately from the table to the kitchen, the application removes physical delays, provides instant synchronization and significantly speeds up service delivery. Simultaneously, the system enforces real-time inventory control by automatically locking dishes when daily quotas are reached, preventing the ordering of out-of-stock items. Crucially, it guarantees billing precision by automatically tracking every separate order placed by a table and consolidating them into a single, error-free invoice, thereby preventing revenue loss and reducing staff workload. This approach eliminates transcription errors, simplifies order management, and ensures a smoother, more efficient operation for both staff and customers.

2.2 Product Features

The system functionality is divided into four major logical modules: Access Control and Administration, Menu and Inventory Management, Order Processing and Service, and Kitchen Operations.

2.2.1 Access Control and Administration

- **User Authentication:** The system secures access through a username and password login mechanism. Each staff member (Chef, Waiter) is assigned a unique account, and the system strictly enforces role-based access control, ensuring users can only access functions relevant to their specific job responsibilities.

- **User Account Management:** The Administrator (Restaurant Owner) retains exclusive administrative privileges to create, edit, or delete accounts for employees. Staff members are restricted to utilizing the system for operational tasks and cannot modify the accounts of others.

2.2.2 Menu and Inventory Management

- **Menu Management:** The system allows for the creation and maintenance of a central database of dishes, where each item includes a name, current price, short description, and notes. The menu can be updated dynamically as needed.
- **Daily Quantity Management:** To prevent over-ordering, the system allows the Chef to input a specific quantity limit for each dish at the start of the day. Once the available quantity reaches zero, the system automatically locks the item, preventing further orders.
- **Daily Menu Replication:** To streamline daily setup, the system provides a "Copy Menu" function. This allows the Chef to clone the menu structure and settings from the previous day to the current day, facilitating quick adjustments to quantities or prices without rebuilding the menu.
- **Dynamic Price Adjustment:** The system supports real-time modification of dish prices, allowing the restaurant to adapt immediately to market changes or specific timing requirements (e.g., holiday pricing).

2.2.3 Order Processing and Service

- **Visual Menu Display:** The application provides a clear, user-friendly interface on mobile devices, enabling customers to browse the menu and input specific preferences or notes for their selected dishes.
- **Order Creation and Submission:** Waitter facilitates the ordering process by logging into the device and handing it to the customer for selection. Once the customer finishes, the waiter reviews, confirms, and submits the order directly to the system.
- **Order Details Management:** The system generates a unique record for every order, capturing the Order ID, list of items, specific quantities, and any accompanying customer notes for accurate tracking.
- **Live Order Modification:** To accommodate customer requests during the meal, the system permits waiters to modify active orders by adding items, removing items, or adjusting quantities before the final bill is settled.
- **Session-Based Billing:** The system manages billing by "Table Session." A session tracks a group of customers from the moment they occupy a table until they leave, automatically aggregating all separate orders placed during that period into a single, unified invoice.

2.2.4 Kitchen Operations

- **Kitchen Order Tracking:** The system provides an interface for the kitchen, displaying a real-time list of confirmed orders sent from the service area. This allows chefs to track requests and prepare dishes in the correct sequence.

2.3 User Classes and Characteristics

2.3.1 Waiter

Description:
Waiters are the primary users of the restaurant management system. They are responsible for assisting customers in selecting dishes and creating food orders using a tablet device. Waiters interact with the system frequently throughout their working shifts and act as the main link between customers and the kitchen.

Set of waiter's product function:

- Log in to the system using assigned credentials.
- Access and display the current menu to customers.
- Create new orders for each table or customer group.
- Can choose serving tables.
- Select food items, enter quantities, and add special notes based on customer requests.
- Submit confirmed orders to the kitchen.
- Modify existing orders (add or remove items) while the table is still active, if permitted.
- Calculate the bill for one order at a table.

System Privileges:

- Authorized to create, edit, and submit orders.
- Limited to order-related functionalities only.
- Not allowed to manage menu items, prices, portion limits, or system configurations.

Frequency of Use:

- Uses the system **frequently and continuously** during working hours.
- Represents the user class with the highest usage frequency.

Technical Expertise:

- Basic level of technical knowledge.
- Familiar with tablet-based touch interactions.
- No advanced technical skills required.

Priority:

- Waiters are considered the **favored user class**.
- The system must be intuitive, fast, and reliable to support efficient customer service.

2.3.2 Chef

Description:
Chefs are users responsible for managing the menu and preparing food within the restaurant management system. Although they do not interact directly with customers, chefs play a critical role in controlling daily menu items, portion availability, and handling orders submitted by Waiters.

Primary Responsibilities:

- Log into the system using a kitchen account.
- View orders submitted by Waiters.
- Review order details, including items, quantities, and special instructions.
- Manage the daily menu by adding or updating menu items.
- Set and adjust available portions for each dish.
- Update item availability when portions run out to prevent further ordering.
- Reset item quantities at the start of each business day.
- Copy the previous day's menu to a new day and adjust quantities as needed.

System Privileges:

- Authorized to manage menu items and portion quantities.
- Authorized to view all incoming orders.
- Not allowed to create customer orders.
- Not allowed to manage user accounts or advanced system configurations.

Technical Expertise:

- Basic to intermediate level of technical proficiency.
- Comfortable with simple management interfaces.
- No advanced IT knowledge required.

Frequency of Use:

- Uses the system **regularly**, especially at the beginning of the day and during peak hours.
- Usage frequency is lower than Waiters but still consistent.

Importance / Priority:

- Chefs are considered an **important user class** (secondary favored user class).
- Menu and portion management features must be clear and accurate to ensure smooth operations.

2.3.3 Admin

Description:
The Admin represents the restaurant owner and is responsible for the overall management of the restaurant management system. This role oversees user management, monitors order activities, and maintains basic system configurations. Admin does not participate in daily ordering operations but holds the highest level of system privileges to ensure stable and correct system operation.

Primary Responsibilities:

- Log into the system with administrative privileges.
- Manage user accounts:
    - Create, update, or deactivate Waiter and Chef accounts.
    - Assign roles and access permissions.
- View an overview of restaurant order activities.
- Monitor order history by date or by table.
- Manage basic restaurant information (restaurant name, number of tables).
- Handle exceptional situations (e.g., correcting orders in case of system errors).

System Privileges / Access Level:

- Full access to all system features.
- Authorized to manage users and permissions.
- Authorized to access menu and order data.
- Does not normally create customer orders during daily operations.

Frequency of Use:

- Uses the system **infrequently**.
- Accesses the system mainly for management and configuration purposes.

Technical Expertise:

- Intermediate level of technical proficiency.

- Capable of using basic administrative functions.
- No advanced system or programming knowledge required.

Importance / Priority:

- Admin is an **important but non-primary user class**.
- Administrative features must be secure, clear, and reliable.

2.4 Operating Environment

Environment of Waiter's Use:

- Operates in a restaurant environment, which may be busy and time-critical.
- The primary device is a tablet provided by the restaurant.

Environment of Chef's Use:

- Operates within the kitchen area of the restaurant.
- The environment may be noisy and time-sensitive, requiring fast and accurate interactions.

Environment of Admin's Use:

- Can access the system from an office or management area.
- Not restricted to kitchen or service environments.

The Restaurant Management System is designed to operate within a real-world restaurant environment where operations are continuous, time-sensitive, and require high reliability. The software must function consistently across multiple devices and coexist peacefully with existing hardware and software components used in the restaurant.

Hardware Platform:

- Tablet devices used by Waiters to create and manage customer orders.
- Desktop computers or laptops used by Chefs and Admins to manage menus, orders, and user accounts.
- A local or cloud-based server responsible for data storage and request processing.
- A stable local area network (LAN) or Internet connection to support communication between client devices and the server.

Operating Systems:

- Tablets running common mobile operating systems such as **Android (version 10 or later)** or **iOS (version 14 or later)**.
- Desktop or laptop computers running **Windows 10 or later**, **macOS**, or common **Linux distributions**.
- The server may operate on a **Linux-based operating system** or **Windows Server**, depending on the deployment strategy.

Software Environment:

- The system is implemented as a web-based or client–server application and is accessible through modern web browsers such as **Google Chrome, Microsoft Edge, Safari, or Firefox**.
- Data is stored in a relational or non-relational database management system (e.g., MySQL, PostgreSQL, or equivalent).
- Server-side software components must support concurrent access from multiple users.

Coexistence with Other Systems:

- The system must operate alongside and not interfere with other software used in the restaurant, such as point-of-sale (POS) systems, accounting software, or standard office applications.
- System resource usage (CPU, memory, network bandwidth) should be optimized to prevent performance degradation of coexisting applications.

Operational Constraints:

- The system must remain stable during peak hours when the volume of orders is high.
- It should tolerate short periods of unstable network connectivity without critical failure.
- The user interface must be optimized for touch-based interaction and high-pressure working environments.

2.5 Design and Implementation Constraints

1. Corporate or regulatory policies: N/A

2. Hardware Limitations

+ The system must run on Tablet devices.
+ Timing & memory requirements: N/A

3. Interfaces to other applications: N/A

4. Specific Technologies, Tools, and Databases

+ The application is web-based, running on the default browser of the tablet (Safari on IOS / Google on Android).

+ There must be a wireless connection between tablets and the server.

5. Parallel Operations

+ The system must support simultaneous access by multiple users (Waiters, Chefs and Admin) each with their own accounts.

+ The system must support concurrency for dish & ingredient quantities; when a dish / ingredient is empty, the system must prevent further action.

6. Language Requirements

+ The UI must support at least Vietnamese and English.

+ The system for employees can be written in only 1 language.

7. Communications Protocols

+ The system uses standard web protocols (HTTPS) for browser-based systems with Internet access.

8. Security Considerations

+ The system must enforce login via account and password for every employee.

+ Access control must be strictly enforced based on roles: Admin (create/delete/modify accounts) and Waiters and Chefs (use system).

9. Design Conventions or Programming Standards: N/A

2.6 User Documentation

1. USER MANUAL

Administration Guide:

+ Instructions on how to create, edit, and delete user accounts.

+ Explanation of role assignment.

Chef Guide:

+ How to copy the menu from the previous day to the current day.

+ How to input and adjust the limited quantity of dishes for the day.

+ How to view the list of incoming orders from the service area.

Service & Order Management Guide:

+ How to use the tablet device to present the menu to customers and input their selections.

+ Steps to confirm and send orders to the system.

+ Instructions on adding/removing items or changing quantities for active orders.

2. SOURCE CODE

+ A README.md file guiding how to install dependencies and run the system locally.

+ A document for deploying the system.

+ A document explaining every component in the system.

3. DELIVERY FORMATS AND STANDARDS

+ The documentation will be delivered as part of the formal "handover documents" package.

+ The documentation will be in Vietnamese and English.

+ The manual must be written clearly for non-technical users.

2.7 Assumptions and Dependencies

1. Assumptions

+ **Scope:** Designed to serve a single restaurant entity.

+ **Menu**: the menu for a new day is largely based on the previous day's menu. The workflow relies on the Chef copying the previous day's menu and making minor adjustments to prices or quantities rather than creating a new menu from scratch every day.

2. Dependencies

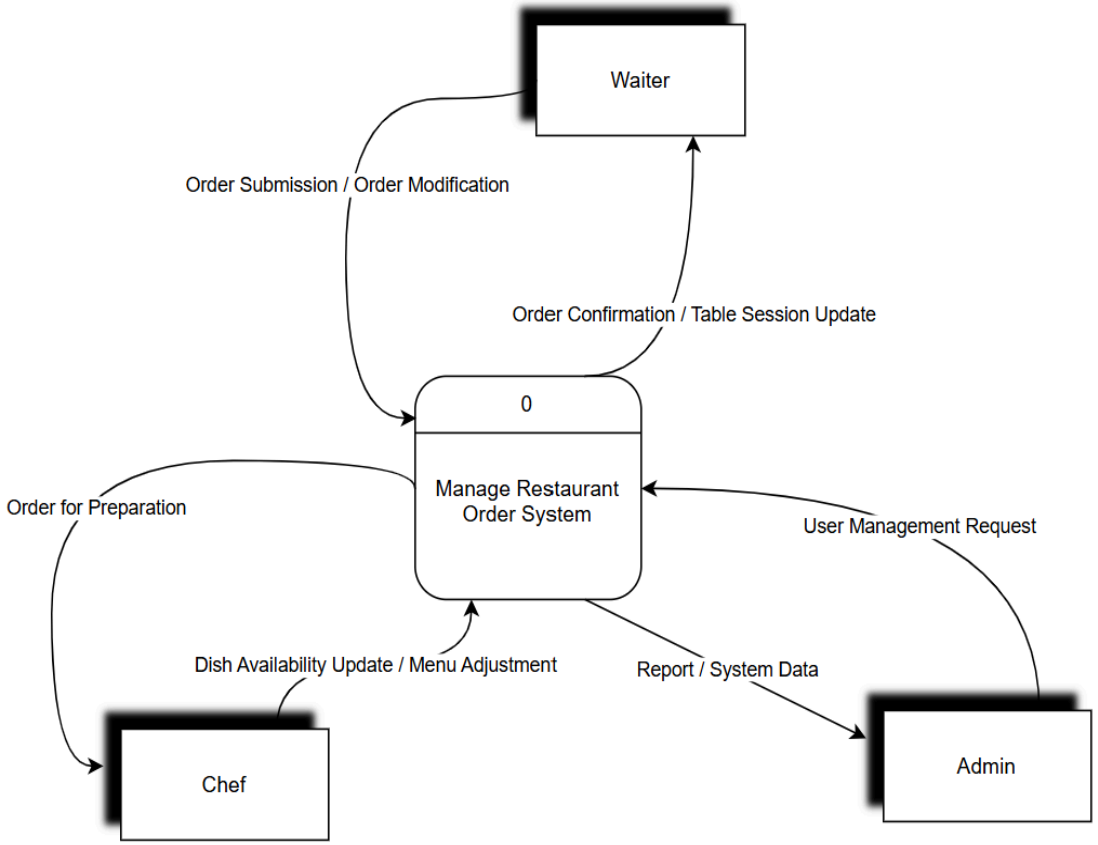
+ **Hardware Availability:** The project is dependent on the client (restaurant) providing tablet devices capable of running a modern web browser. The software is not responsible for hardware compatibility beyond standard browser support.

+ **Network Infrastructure:** The system functionality is strictly dependent on the availability of a stable Internet connection at the restaurant premises.

+ **Browser Environment:** The application depends on a standard web browser environment (e.g., Chrome, Safari)

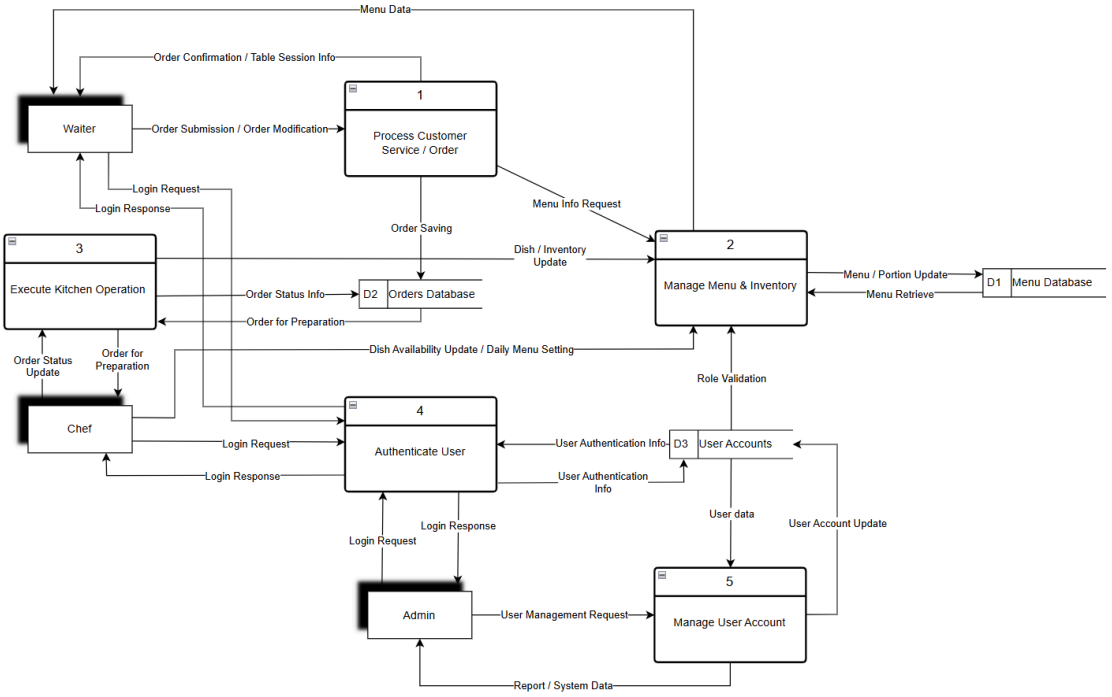
3. Data Flow Diagrams

3.1 Context Diagram



The Context Diagram illustrates the top-level view of the Manage Restaurant Order System process and its interactions with the three primary user roles: Waiter, Chef, and Admin. Waiters submit orders, make modifications, and receive confirmations and table session updates. Chefs receive orders for preparation and provide dish availability updates or menu adjustments back to the system. Admins manage user accounts and access system reports. This diagram clearly defines system boundaries, showing how data flows between the Restaurant Management System and its users, without detailing internal processes or data stores.

3.2 Diagram 0



The Diagram 0 represents the explosion of the Context Diagram for the Restaurant Management System, illustrating the major processes, data stores, and the flow of data between them. It shows a unified view of the system where all user interactions and data flows are connected in a single, coherent block.

External Entities:

- Waiter: Interacts with the system to submit orders or modify existing orders. Receives order confirmations and table session updates from the system.
- Chef: Receives orders for preparation from the system and provides dish availability updates.
- Admin: Handles user management requests, receives reports, and performs login operations through user authentication processes.

Processes:

1. Process Customer Service / Order (P1):
    - Receives order input from Waiter.
    - Saves orders to the Orders Database (D2).
    - Sends order confirmations and session updates back to Waiter.
    - Requests menu information from Manage Menu & Inventory (P2) as needed.

2. Manage Menu & Inventory (P2):
    - Retrieves menu details from Menu Database (D1).
    - Updates menu items and portions as needed.
    - Receives requests from Process Customer Service / Order (P1) and updates inventory if necessary based on orders.
3. Execute Kitchen Operation (P3):
    - Reads orders from Orders Database (D2).
    - Sends orders to Chef for preparation.
    - Receives dish availability updates from Chef.
    - Updates the Orders Database (D2) with order status.
    - Updates sent to Manage Menu & Inventory (P2) if inventory adjustments are required.
4. Authenticate User (P4):
    - Processes login requests from users.
    - Authenticates users via User Accounts (D3).
    - Provides authentication information to Manage User Accounts (P5).
5. Manage User Account (P5):
    - Receives user management requests from Admin.
    - Updates user account information in User Accounts database (D3).
    - Sends reports back to Admin.

Data Stores:

- D1: Menu Database: Stores all menu items, descriptions, prices, and portion counts.
- D2: Orders Database: Stores all orders, including items, quantities, and table session information.
- D3: User Accounts: Stores account credentials and role-based access information for Waiters, Chefs, and Admin.

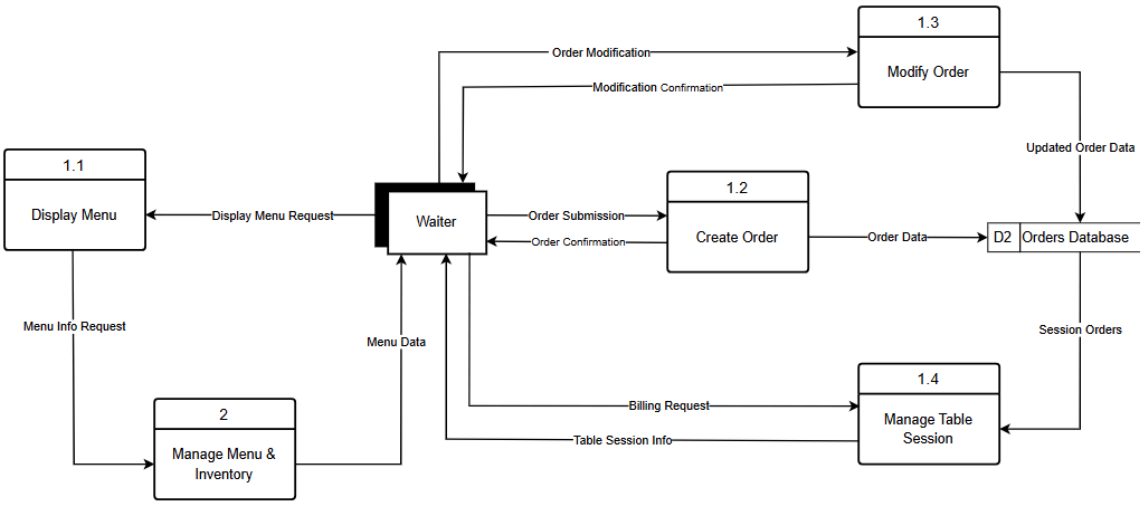
Data Flow Highlights:

- All processes are connected to their relevant data stores and users.

- P1, P2, and P3 form the order and kitchen management flow, while P4 and P5 handle user authentication and administration.
- Data flows from users to processes, between processes and data stores, and back to users, reflecting the full functional logic of the Restaurant Management System.

3.3 Child Diagrams

3.3.1 Process 1: Process Customer Service / Order



This process handles the complete lifecycle of customer orders for the waiters. It synchronizes real-time menu data and manages active table records within the database to ensure accurate order submission, dynamic modifications, and seamless final billing.

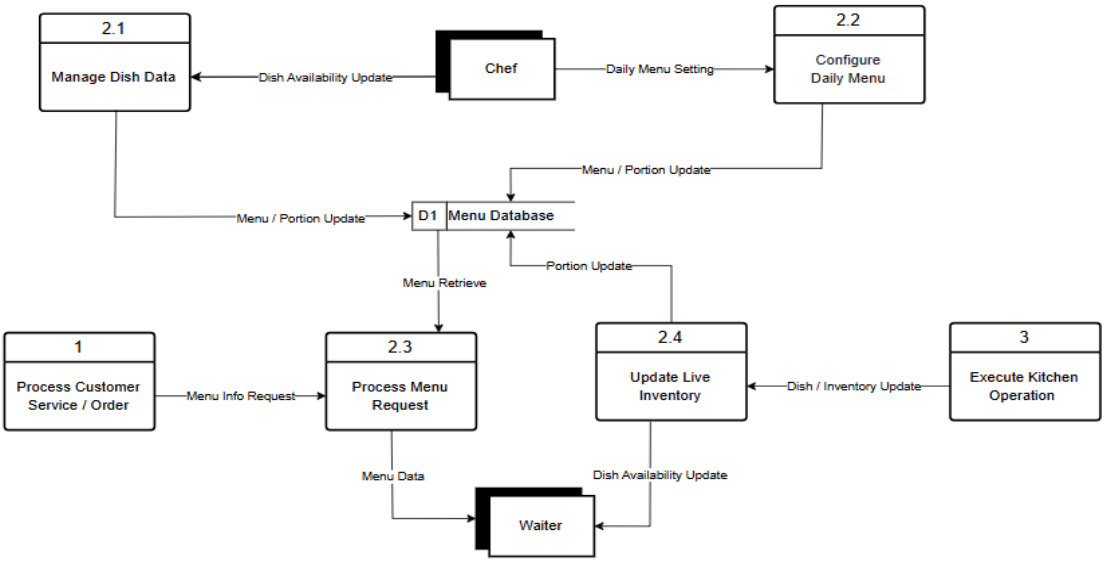
1.1 Display Menu: Handles the initial menu viewing. When Waiter requests the menu, the system fetches the current dish and pricing data from the Menu & Inventory module and displays it on the Waiter's screen.

1.2 Create Order: Manages new dining requests. It receives the customer's selections from the Waiter, saves the new order details into the D2 (Orders Database), and sends a success confirmation back to the Waiter's device.

1.3 Modify Order: Handles mid-meal changes, such as adding or canceling items. When the Waiter submits an update, the system adjusts the existing records in the D2 (Orders Database) and confirms the modification.

1.4 Manage Table Session: Handles the final checkout process. When the Waiter requests the bill, the system gathers all the orders tied to that specific table from the D2 (Orders Database), calculates the total amount, and displays the final invoice for payment.

3.3.2 Process 2: Manage Menu & Inventory



This process acts as the centralized configuration and inventory control hub for the restaurant, handling the complete management of dish data and daily availability limits for the kitchen staff. It maintains the core menu database and synchronizes live portion updates across the system to ensure accurate ordering and actively prevent out-of-stock scenarios.

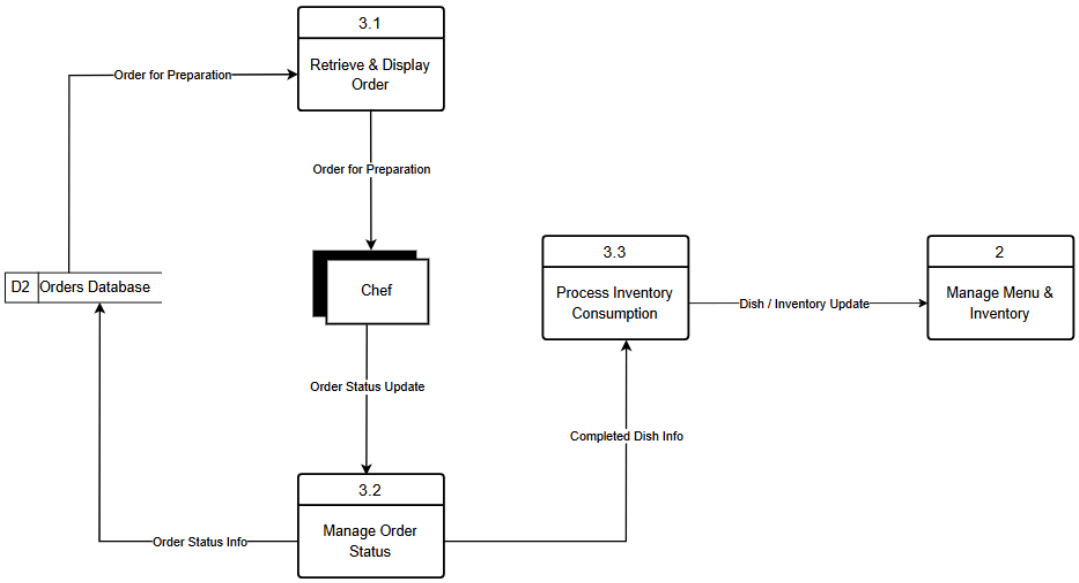
2.1 Manage Dish Data: Handles the core dish information. When the Chef makes adjustments to a dish's details or overall availability, the system saves these changes into the D1 (Menu Database) to keep the restaurant's central catalog current.

2.2 Configure Daily Menu: Manages the daily setup of the restaurant. When the Chef configures the settings for the day (such as setting specific daily stock limits), the system applies these portion rules directly to the D1 (Menu Database) so the front-of-house staff knows exactly what can be sold.

2.3 Process Menu Request: Acts as the information bridge for the waiters. When it receives a request from the customer order system (Process 1), it fetches the most up-to-date menu from the D1 (Menu Database) and sends the data straight to the Waiter's screen.

2.4 Update Live Inventory: Handles real-time stock tracking. When the kitchen finishes preparing a dish (triggered by Process 3), the system automatically deducts that portion from the D1 (Menu Database) and instantly alerts the Waiter's device if an item goes out of stock.

3.3.3 Process 3: Execute Kitchen Operation



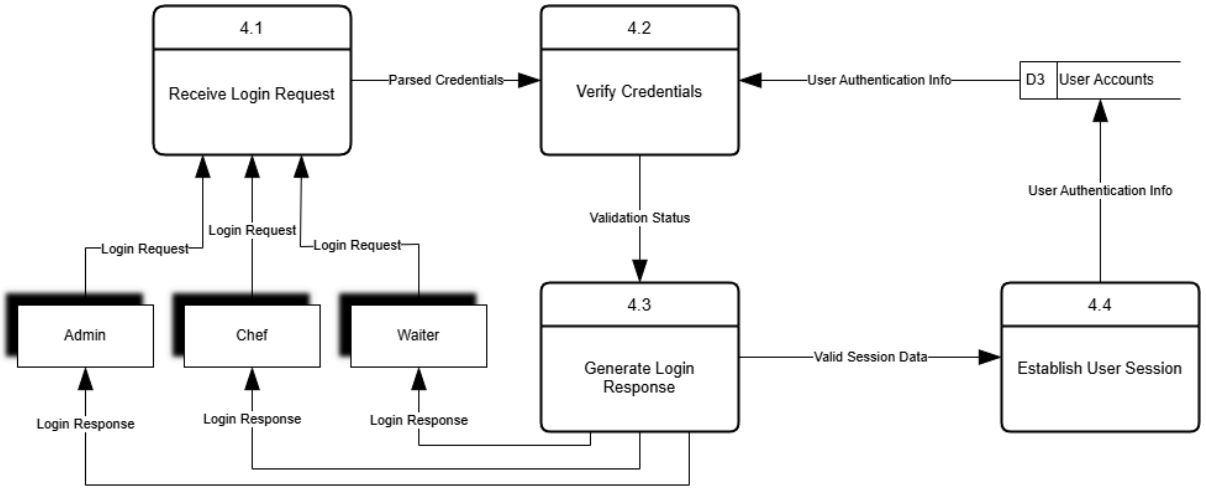
This process acts as the back-of-house workflows, handling the real-time tracking of incoming customer orders for the chefs. It retrieves pending preparation tasks from the database, processes live status updates, and triggers automatic inventory consumption to ensure streamlined food delivery and accurate stock synchronization.

3.1 Retrieve & Display Order: Handles the kitchen's active order queue. It continuously pulls new preparation requests from the D2 (Orders Database) and displays them on the Chef's screen so the kitchen knows exactly what to cook next.

3.2 Manage Order Status: Tracks the progress of food preparation. When the Chef taps their screen to mark an order as started or finished, the system saves this updated status back to the D2 (Orders Database) and forwards the completion details to the inventory tracker.

3.3 Process Inventory Consumption: Manages automatic stock deduction. Whenever it receives a notification from the status manager that a specific dish is finished, it sends an update to the Menu & Inventory module so the daily available portions can be reduced.

3.3.4 Process 4: Authenticate User



This process acts as the centralized security gateway for the entire system, handling login requests for Waiters, Chefs, and Admins. It verifies credentials and manages active sessions to enforce strict security timeouts.

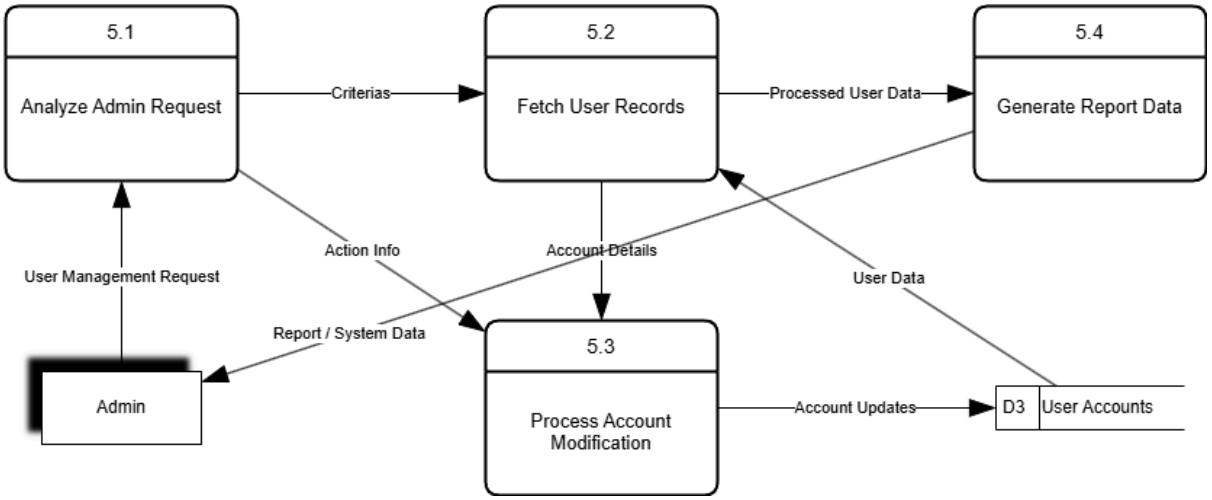
4.1 Receive Login Request: Captures the incoming Login Request (username and password) from the Waiter, Chef, or Admin.

4.2 Verify Credentials: Retrieves User Authentication data (password hashes and role assignments) from D3 (User Accounts) to securely verify the provided credentials.

4.3 Generate Login Response: Generates the authentication result (success or failure) and sends the Login Response (which includes the session access token) back to the specific user.

4.4 Establish User Session: If validation is successful, this sub-process generates a timestamped session record and sends the User Authentication Info to D3 (User Accounts) to enforce inactivity timeouts (10 minutes for Admin, 15 minutes for staff).

3.3.5 Process 5: Manage User Account



This process handles the restaurant owner's administrative duties, allowing the Admin to maintain staff accounts, enforce role-based access control, and generate system overview reports.

5.1 Analyze Admin Request: Receives the User Management Request from the Admin and determines whether the action is an account modification (create/edit/delete) or a reporting query.

5.2 Fetch User Records: Queries D3 (User Accounts) to retrieve the necessary User Data, such as existing account details or a list of currently logged-in users.

5.3 Process Account Modifications: Processes the specific account changes and sends a User Account Update flow to D3 (User Accounts) to securely store the new or updated Waiter and Chef credentials.

5.4 Generate Report Data: Formats the retrieved database information into structured Report / System Data (like daily summaries or user activity logs) and delivers it to the Admin.

4. System Features

4.1 System Feature 1: Waiter Order Processing and Table Session Management

4.1.1 Description and Priority

This feature provides all functions required by the **Waiter** role to support staff-assisted digital ordering. It covers presenting the menu to customers, creating and submitting orders, and aggregating orders by **table session** to produce a unified view of all items ordered during a meal. The feature also ensures that out-of-stock dishes cannot be ordered by enforcing real-time portion availability.

Priority: High (core functionality of the order management subsystem and the primary workflow used throughout operating hours).

4.1.2 Stimulus/Response Sequences

- Waiter logs in using assigned credentials → System authenticates the account and displays the ordering interface.
- Waiter selects a serving table / opens an active table session → System loads the table session and displays current session orders (if any).
- Waiter opens the menu on the tablet for customers → System displays the latest menu with current prices and availability status (available/locked).
- Customer selections (assisted by Waiter)→ System validates input and temporarily stores selections.
- Waiter reviews and confirms the order → System checks dish portion availability and creates a new Order ID.
- Waiter submits the confirmed order → System saves the order, updates the table session totals, deducts portions, and sends the order to the kitchen list.
- A dish reaches zero remaining portions while ordering → System locks the dish immediately and prevents further selection, displaying an out-of-stock message.

4.1.3 Functional Requirements

REQ-W1: The system allows Waiter to log in using assigned credentials before accessing any order-related functions.

REQ-W2: The system allows Waiter to select a serving table and either create a new table session (if vacant) or access an existing active table session.

REQ-W3: The system displays the current menu to the Waiter/customer, including dish name, price, short description, and availability status (available/locked).

REQ-W4: The system allows Waiter to create a new order by selecting dishes, specifying quantities, and attaching optional notes for each dish.

REQ-W5: The system validates order input such that dish quantity must be a positive integer and dish notes must not exceed a configured maximum length (TBD characters).

REQ-W6: The system generates a unique Order ID for each confirmed order and stores the order details (items, quantities, notes, table session ID, timestamp) in the Orders Database.

REQ-W7: The system allows the Waiter to submit a confirmed order to the kitchen, and the order becomes visible in the kitchen order list immediately after submission.

REQ-W8: The system checks dish portion availability at the time of order confirmation/submission and prevents submission if any selected dish is unavailable or locked.

REQ-W9: When an order is successfully confirmed, the system deducts the corresponding dish portions from the daily quantity count in an atomic manner to prevent negative inventory under concurrent submissions.

REQ-W10: If a dish's remaining portion reaches zero, the system locks the dish and prevents the Waiter/customer from selecting it in subsequent ordering actions.

REQ-W11: The system displays a clear error message when an order cannot be submitted due to invalid input, unavailable dishes, or insufficient remaining portions.

REQ-W12: In the event of a short network disconnection during order submission, the system caches the pending order locally (offline queue) and automatically synchronizes it when the connection is restored, ensuring no confirmed order is lost.

4.2 System Feature 2: Live Order Modification

4.2.1 Description and Priority

This feature allows waiters to modify an existing order associated with a table session while the session is still active. During a dining session, customers may request additional items, removal of previously ordered items, or adjustments to item quantities.

The system supports these changes by updating the order record and verifying item availability when additional items are added. When a modification is confirmed, the system updates the stored order information, recalculates the order total, and ensures that the kitchen receives the updated order details.

This feature improves flexibility during the dining process and ensures that order information remains accurate throughout the session.

Priority: High (required to support real-time order adjustments during active dining sessions).

4.2.2 Stimulus / Response Sequences

- Waiter opens the active order associated with a table session → System displays the current order details for the selected session.
- Waiter modifies the order by adding items, removing items, adjusting quantities, and updating notes → System verifies item availability for any newly added item.
- The system updates the stored order record.
- The system recalculates the order total for the session.
- The system transmits the updated order information to the kitchen.

4.2.3 Functional Requirements

REQ-L1: The system allows waiters to view active orders associated with a table session.

REQ-L2: The system allows waiters to modify an existing order while the table session is active.

REQ-L3: The system allows waiters to add menu items to an existing order.

REQ-L4: The system allows waiters to remove previously ordered items from an order.

REQ-L5: The system allows waiters to adjust the quantity of ordered items.

REQ-L6: The system verifies item availability when items are added to an order.

REQ-L7: The system updates the stored order record after modifications are confirmed.

REQ-L8: The system automatically recalculates the order total after modifications.

REQ-L9: The system transmits updated order information to the kitchen for preparation.

4.3 System Feature 3: Session-Based Billing

4.3.1 Description and Priority

This feature manages billing based on table sessions. A table session begins when customers occupy a table and ends when payment is completed and the table becomes available for new customers.

All orders placed during the dining session are associated with the same table session. When billing is requested, the system retrieves all orders associated with the session and aggregates them into a single bill. The system then calculates the total amount and generates the final invoice for payment.

This feature ensures that all orders are accurately recorded and combined into a single bill, simplifying payment processing for both customers and restaurant staff.

Priority: High (required for accurate billing and table management).

4.3.2 Stimulus / Response Sequences

- Waiter opens an active table session when customers request the bill → System retrieves all orders associated with the table session.
- Waiter initiates the billing process → System aggregates all orders recorded within the table session.
- The system calculates the bill for the session by summing the prices and quantities of all ordered items.
- The system generates the final invoice containing the list of ordered items, quantities, and total amount.
- Waiter confirms payment completion → System closes the table session and updates the table status to available for the next customers.

4.3.3 Functional Requirements

REQ-B1: The system creates a table session when customers occupy a table.

REQ-B2: The system associates all orders with the corresponding table session.

REQ-B3: The system retrieves all orders associated with a table session when billing is requested.

REQ-B4: The system aggregates all orders within a session into a single bill.

REQ-B5: The system calculates the bill for the table session.

REQ-B6: The system generates a final invoice for payment.

REQ-B7: The system closes the table session after payment is completed.

REQ-B8: The system marks the table as available after the session is closed.

4.4 System Feature 4: Chef Menu, Inventory and Kitchen Operations

4.4.1 Description and Priority

This feature provides all functions required by the Chef role: daily menu setup, portion quantity control, real-time kitchen order tracking, and menu adjustments. It ensures the kitchen always works with accurate, up-to-date information and prevents over-ordering of out-of-stock dishes.

Priority: High (directly impacts service speed and prevents revenue loss).

4.4.2 Stimulus/Response Sequences

- Chef logs in with a kitchen account → System displays current day's menu and real-time list of pending orders.
- Chef clicks "Copy Menu from Previous Day" → System automatically clones all dishes, prices, descriptions; Chef adjusts quantities/prices as needed.
- Chef sets daily portion limit for a dish → System saves limit and automatically locks the dish when quantity reaches zero.
- Waiter submits an order → System instantly pushes the order to Chef's kitchen screen with Order ID, items, quantities, and special notes.
- Chef updates order status to "In Progress" or "Completed" → System notifies the corresponding Waiter and updates table session.
- Chef modifies price or description of a dish → System updates the menu in real time for all active tablets.

4.4.3 Functional Requirements

REQ-C1: The system allows the Chef to copy the entire previous day's menu (dishes, prices, descriptions, notes) to the current day with one action.

REQ-C2: The system allows the Chef to create, edit, delete, or update any dish (including name, price, description).

REQ-C3: The system enables the Chef to set, modify, and view daily portion quantities for each dish.

REQ-C4: The system automatically locks a dish and notify all Waiter tablets in real time when the remaining portion reaches zero.

REQ-C5: The system allows the Chef to manually mark any dish as unavailable (even if portions remain) for special reasons.

REQ-C6: The system provides a real-time kitchen dashboard showing all confirmed orders with Order ID, table number, items, quantities, special notes, and status.

REQ-C7: The system allows the Chef to update order status to "In Progress", "Completed", or "Cancelled" and immediately notify the corresponding Waiter.

REQ-C8: The system supports real-time price and description modification by the Chef.

REQ-C9: The system provides search/filter functions for dishes (by name, category) and orders (by table or status).

REQ-C10: The system supports concurrent access by multiple Chefs with proper locking to prevent negative inventory.

REQ-C11: The system automatically deducts portion counts when an order is confirmed by Waiter.

4.5 System Feature 5: System Administration

4.5.1 Description and Priority

This feature allows the Admin (restaurant owner) to manage user accounts, view overall system reports, and configure basic restaurant information. It ensures secure role-based access and system oversight without interfering with daily operations.

Priority: High (critical for security and user account lifecycle management).

4.5.2 Stimulus/Response Sequences

- Admin logs in with administrative privileges → System displays user management dashboard and overview reports.
- Admin creates a new Waiter or Chef account → System assigns username, password, and role; account is immediately active.
- Admin edits/deactivates a user account → System updates permissions and logs the change.
- Admin views order history → System shows summary by date or by table (total orders, revenue overview).
- Admin updates basic restaurant settings (name, number of tables) → Changes are saved and reflected system-wide.

4.5.3 Functional Requirements

REQ-A1: The system allows only the Admin to create, edit, deactivate, delete, or reset password for Waiter and Chef accounts.

REQ-A2: The system enforces strict Role-Based Access Control (RBAC) following the least-privilege principle.

REQ-A3: The system provides the Admin with an overview dashboard including daily order summary, table occupancy, and order history (filterable by date or table).

REQ-A4: The system allows the Admin to configure basic restaurant information (name, number of tables).

REQ-A5: The system records all administrative actions in an audit log including timestamp, user ID, action type, and IP address.

REQ-A6: The system terminates inactive sessions and requires re-authentication.

REQ-A7: The system allows the Admin to export order reports (PDF/Excel) for daily or monthly summary.

REQ-A8: The system allows the Admin to view currently logged-in users and force logout if necessary.

REQ-A9: The system prevents the Admin from creating or modifying customer orders during daily operations.

REQ-A10: The system requires authentication for all staff users (Waiter, Chef, Admin) before accessing protected functions.

REQ-A11: The system denies access and displays an “Access Denied” message if a user attempts to access a function outside their role.

REQ-A12: The system prevents login for deactivated accounts.

REQ-A13: The system validates authorization on the server side for every protected request.

5. Nonfunctional Requirements

5.1 Performance Requirements

- The average response time for all actions (submit order, update menu, change kitchen status) shall be ≤ 1.5 seconds, with 95% of requests under normal load.
- The system supports at least 30 concurrent users (15–20 Waiters + 3–5 Chefs + 1 Admin) without performance degradation.
- Real-time notifications (kitchen screen, table session updates) shall achieve a latency of ≤ 3 seconds (using WebSocket or long polling).
- The system handles peak load of 60 orders per hour during peak hours (Friday–Sunday evenings) without database locking exceeding 2 seconds.
- Inventory updates are synchronized immediately (using row-level locking or optimistic concurrency control).

5.2 Availability and Reliability Requirements

- The system shall achieve an uptime of ≥ 99.5% during restaurant operating hours (10:00–22:00).
- Mean Time To Recovery (MTTR) shall be ≤ 5 minutes.
- In the event of network disconnection, tablets support offline mode (local order caching) and automatically synchronize when the connection is restored (no order shall be lost).
- The system includes a failover mechanism (redundant server or cloud auto-scaling) to avoid single points of failure.
- In the event of power loss or server crash, the system recovers the exact state of orders and inventory up to the most recent point.

5.3 Safety Requirements

- The system mandatorily prevents any order when the remaining portion of a dish reaches zero (no over-selling allowed).
- All data (orders, menu, user accounts) is backed up hourly (incremental) and once daily (full backup).
- All administrative actions (create/delete accounts, change prices, copy menu) are recorded in an audit log (timestamp, user ID, action, IP) and retained for at least 30 days.

5.4 Security Requirements

- All users authenticate using username and password (passwords hashed with bcrypt or Argon2, minimum 8 characters including uppercase, lowercase, numbers, and special characters).
- Session timeout occurs after 10 minutes of inactivity for Admin accounts and 15 minutes for Chef/Waiter accounts.
- Role-Based Access Control (RBAC) is strictly enforced following the principle of least privilege.
- Rate limiting is applied to login attempts (maximum 5 attempts per minute per account) to prevent brute-force attacks.
- Sensitive data (passwords) is encrypted at rest.
- The system does not store any personal customer data (name, phone, payment information) unless explicitly required for loyalty programs in future phases.

5.5 Usability Requirements

- The tablet interface follows touch-first principles: every primary action (browse menu, submit order) shall require ≤ 3 taps.
- User error rate (incorrect input or mistaken submission) shall be less than 1% after one week of use.
- The system supports both Vietnamese and English, with font size ≥ 16 pt and high contrast (WCAG AA) to ensure readability for older staff.
- Training time for new staff shall be ≤ 30 minutes for both Chefs and Waiters.

5.6 Maintainability and Portability Requirements

- The code is designed in a modular structure (separate modules: Authentication, Menu-Inventory, Order, Admin) with unit test coverage ≥ 90%.
- The system runs on all modern browsers (Chrome 110+, Safari 16+, Edge) on Android 10+ and iOS 14+ without requiring a native app installation.
- Documentation (README, API docs, deployment guide) is kept up-to-date and stored in the repository.

5.7 Interoperability and Compliance Requirements

- The system operates concurrently with existing restaurant POS or accounting software without conflicts (no shared ports, no performance impact on other applications).
- The system complies with basic Vietnamese personal data protection regulations (no unnecessary storage of customer information).

6. Other Requirements

6.1 Database Requirements

- The database must support **row-level locking** and **optimistic concurrency control** for the dish quantity field to prevent race conditions when multiple Waiters submit orders simultaneously.
- All critical tables (Menu, Orders, UserAccounts) include appropriate indexes on frequently queried fields (DishID, TableSessionID, OrderID).
- Daily backups (full + incremental) are scheduled automatically at 02:00 AM and stored both on the local server and in cloud storage (Google Drive / AWS S3) for 30 days.

6.2 Deployment and Installation Requirements

- Installation package must include:
    - Docker Compose file for easy local deployment (development & production).
    - Step-by-step deployment guide (README.md).
    - Database initialization script (schema + sample data).
- Minimum server specifications: 4 CPU cores, 8 GB RAM, 100 GB SSD (for 1 restaurant with < 30 tables).

6.3 Reuse Objective Requirements

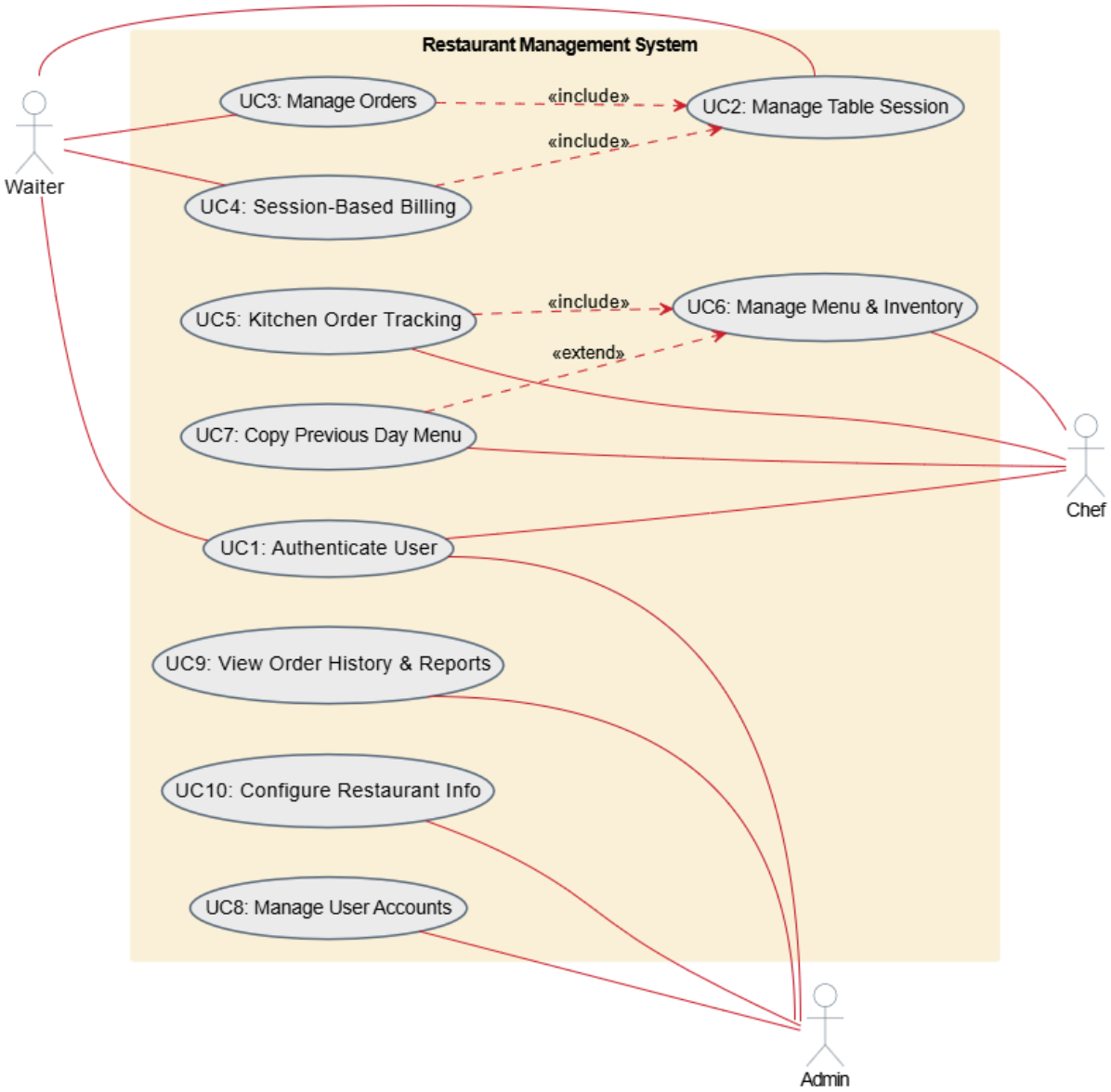
- The User Authentication and RBAC module is designed for reuse in future expansions (e.g., Payment Module, Loyalty Program, or Multi-branch version).
- The real-time notification component (WebSocket) is reusable for any future kitchen status updates or customer-facing display screens.

6.4 Documentation Requirements

- Full user manuals (Administration Guide, Chef Guide, Waiter Guide) are delivered in both Vietnamese and English (PDF + online HTML version).
- Source code documentation includes:
    - README.md with installation and run instructions.
    - API documentation (Swagger/OpenAPI if applicable).
    - Component diagram explanation for future maintenance.

7. System Analysis

7.1 Use case Diagrams



The system identifies three primary actors with different roles and privilege levels within the Restaurant Management System. All actors are required to authenticate securely before accessing their specific functionalities:

- **Waiter:** An authorized staff member responsible for front-of-house operations and customer service. They interact with the system to open and manage table sessions, create and submit customer orders, and generate session-based billing.
- **Chef:** An authorized kitchen staff member responsible for back-of-house operations. Their main capabilities include tracking and updating the status of kitchen orders, managing daily

menus and inventory portions, and optionally copying previous day menus for quick daily setup.

- **Admin (Restaurant Manager / System Administrator):** A privileged user responsible for managing the restaurant's operational data and supervising the overall business processes. They manage staff user accounts, configure core restaurant information, and view order history and business reports.

7.2 Use case Scenarios

UC1: Authenticate User

Field	Detail
Use case ID	UC1
Use case name	Authenticate User
Area	Security / Access Control System
Actor(s)	Waiter, Chef, Admin
Description	Allows authorized staff to securely log into the system and create a valid role-based session.
Triggering Event	The user navigates to the login screen, enters credentials, and clicks Login.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. The actor enters username and password and clicks Login. 2. System checks the input format. 3. System verifies credentials against User Accounts. 4. System determines the user's role (Waiter/Chef/Admin). 5. System creates a session/token and logs the start time. 6.System returns the login result and redirects to the role dashboard.
Preconditions	Account exists in User Accounts (D3) and is not deactivated.
Postconditions	The user is successfully authenticated and routed to the correct screen.

Assumptions	Session timeout rules apply (Admin 10 mins, Chef/Waiter 15 mins).
Requirements Met	Secure system access based on user roles.
Outstanding Issues	What is the exact lockout duration for exceeding the rate limit?
Priority	High
Risk	High

UC2: Manage Table Session

Field	Detail
Use case ID	UC2
Use case name	Manage Table Session
Area	Order Management Subsystem
Actor(s)	Waiter
Description	Allows waiters to open, select, or close a table session to link incoming orders to a specific table.
Triggering Event	Waiter selects a specific table from the floor plan or list interface.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. The waiter opens the table list screen. 2. System displays table statuses (Vacant / Active). 3. The waiter selects a table. 4. If Vacant: System creates a new session. 5. If Active: System loads current session info. 6. System displays session info (orders, status) to the waiter. 7. The waiter clicks "Close Session" (after customer payment).

	8. System closes the session and updates the table to Available.
Preconditions	Waiter is successfully logged into the system.
Postconditions	Table session is opened, active, or closed properly.
Assumptions	Payment processing is out of scope; system only records "closed".
Requirements Met	Organize orders systematically by table.
Outstanding Issues	None
Priority	High
Risk	Medium

UC3: Manage Orders

Field	Detail
Use case ID	UC3
Use case name	Manage Orders
Area	Order Management Subsystem
Actor(s)	Waiter
Description	Allows waiters to view the menu, create/modify orders, and submit them to the kitchen queue.
Triggering Event	The waiter starts taking orders for an active table session.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Waiter selects View Menu. 2. System retrieves menu (D1) and shows availability. 3. Waiter selects dishes, quantities, and inputs notes.

	4. System validates inputs. 5. Waiter clicks Confirm. 6. System checks portion/availability before confirming. 7. System generates OrderID and saves to Orders (D2). 8. System deducts portions atomically. 9. System forwards order to the Kitchen dashboard. 10. System returns Order Confirmation to Waiter.
Preconditions	Waiter is logged in; Table session is active.
Postconditions	Order is saved, sent to kitchen, and inventory is accurately deducted.
Assumptions	Kitchen receives orders in real-time.
Requirements Met	Digital order processing and real-time inventory deduction.
Outstanding Issues	None
Priority	High
Risk	High

UC4: Session-Based Billing

Field	Detail
Use case ID	UC4
Use case name	Session-Based Billing
Area	Order Management Subsystem
Actor(s)	Waiter
Description	Aggregates all orders in a table session to calculate the total and generate an invoice.

Triggering Event	Customer requests the bill.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Waiter selects View Table Session Summary. 2. System retrieves all orders for the session (D2). 3. System aggregates items and calculates the total. 4. System displays the invoice preview. 5. Waiter confirms "Payment completed". 6. System closes the table session. 7. System updates table status to Available.
Preconditions	Waiter is logged in; Table session is active.
Postconditions	Invoice generated, session closed, and table freed.
Assumptions	External payment gateway is not integrated.
Requirements Met	Accurate calculation of customer bills.
Outstanding Issues	None
Priority	High
Risk	Medium

UC5: Kitchen Order Tracking

Field	Detail
Use case ID	UC5
Use case name	Kitchen Order Tracking
Area	Kitchen Management Subsystem
Actor(s)	Chef

Description	Allows the Chef to view the incoming order queue and update food preparation statuses.
Triggering Event	Chef checks the dashboard or a new order arrives.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Chef opens the Kitchen Dashboard. 2. System loads new order list from Orders DB (D2). 3. Chef selects an order to process. 4. Chef updates status (Pending - In Progress - Completed/Cancelled). 5. System saves the new status to D2. 6. System notifies the Waiter of the status update. 7. System updates inventory/portions (via UC6).
Preconditions	Chef is logged in.
Postconditions	Order status updated; Waiter notified and inventory updated.
Assumptions	System UI refreshes automatically without manual page reloads.
Requirements Met	Seamless communication between Waiters and Kitchen.
Outstanding Issues	None
Priority	High
Risk	Medium

UC6: Manage Menu & Inventory

Field	Detail
Use case ID	UC6
Use case name	Manage Menu & Inventory
Area	Kitchen Management Subsystem
Actor(s)	Chef
Description	Allows the Chef to manage dish details, daily portions, and availability status.
Triggering Event	Chef navigates to the Menu & Inventory module.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Chef opens Menu & Inventory Management. 2. System displays current dish list (D1). 3. Chef adds, edits, or deletes dish details. 4. Chef sets daily portions and availability status. 5. System validates data. 6. System saves changes to Menu DB (D1). 7. System updates availability status to Waiters.
Preconditions	Chef is logged in.
Postconditions	Menu and inventory are updated and synced system-wide.
Assumptions	None
Requirements Met	Dynamic menu and daily stock management.
Outstanding Issues	None
Priority	High
Risk	Low

UC7: Copy Previous Day Menu

Field	Detail
Use case ID	UC7
Use case name	Copy Previous Day Menu
Area	Kitchen Management Subsystem
Actor(s)	Chef
Description	Copies the previous day's menu to the current day for rapid system setup.
Triggering Event	Chef clicks the "Copy Menu" button.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Chef selects "Copy Previous Day Menu". 2. System prompts for source date confirmation. 3. System retrieves previous menu data. 4. System clones dishes and prices to the current date. 5. Chef adjusts portions/prices if necessary. 6. System saves to D1 and syncs for Waiters.
Preconditions	Chef is logged in; A previous menu exists in the database.
Postconditions	Today's menu is populated.
Assumptions	This is an optional extension of UC6.
Requirements Met	Reduce manual data entry for kitchen staff.
Outstanding Issues	None
Priority	Medium
Risk	Low

UC8: Manage User Accounts

Field	Detail
Use case ID	UC8
Use case name	Manage User Accounts
Area	Administration Subsystem
Actor(s)	Admin
Description	Allows Admins to create, edit, deactivate, or reset passwords for staff accounts.
Triggering Event	Admin accesses the User Management screen.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Admin opens Manage User Accounts. 2. System displays user list and statuses. 3. Admin selects an action (Create/Edit/Deactivate/Delete). 4. Admin inputs user data (username, role, password). 5. System validates inputs against policies. 6. System saves changes to User Accounts (D3). 7. System records an audit log.
Preconditions	Admin is logged in with appropriate privileges.
Postconditions	User accounts are updated successfully with an audit trail.
Assumptions	Passwords are encrypted before saving.
Requirements Met	System security and staff lifecycle management.
Outstanding Issues	None
Priority	High
Risk	High

UC9: View Order History & Reports

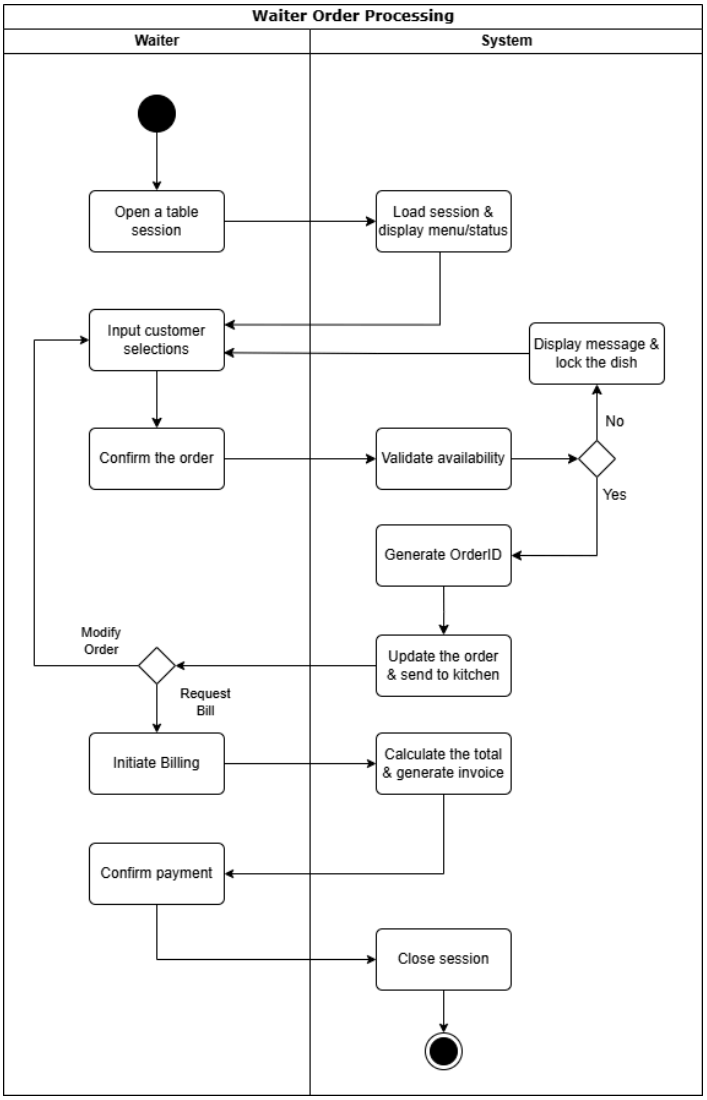
Field	Detail
Use case ID	UC9
Use case name	View Order History & Reports
Area	Administration Subsystem
Actor(s)	Admin
Description	Allows Admins to view order history by specific filters and export reports.
Triggering Event	Admin clicks on the Reports module.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Admin opens Order History & Reports. 2. Admin applies filters (date range, table, status). 3. System queries data from Orders DB (D2). 4. System displays summary and detail results. 5. Admin selects Export (PDF/Excel). 6. System generates file and provides download link.
Preconditions	Admin is logged in.
Postconditions	Report is displayed and/or downloaded.
Assumptions	None
Requirements Met	Business tracking and performance analysis.
Outstanding Issues	None
Priority	Medium
Risk	Low

UC10: Configure Restaurant Info

Field	Detail
Use case ID	UC10
Use case name	Configure Restaurant Info
Area	Administration Subsystem
Actor(s)	Admin
Description	Allows the Admin to configure core system settings such as restaurant name and total table count.
Triggering Event	Admin accesses the System Settings module.
Trigger type	[X] External [ ] Temporal
Steps Performed (Main Path)	1. Admin opens Configure Restaurant Info. 2. System displays the current configuration. 3. Admin edits details (name, table count). 4. System validates inputs. 5. System saves the new configuration. 6. System updates the table display for Waiters.
Preconditions	Admin is logged in.
Postconditions	Core settings applied system-wide.
Assumptions	Waiter devices will sync the new table count immediately.
Requirements Met	System scalability and adaptability.
Outstanding Issues	None
Priority	Low
Risk	Low

7.3 Activity Diagrams

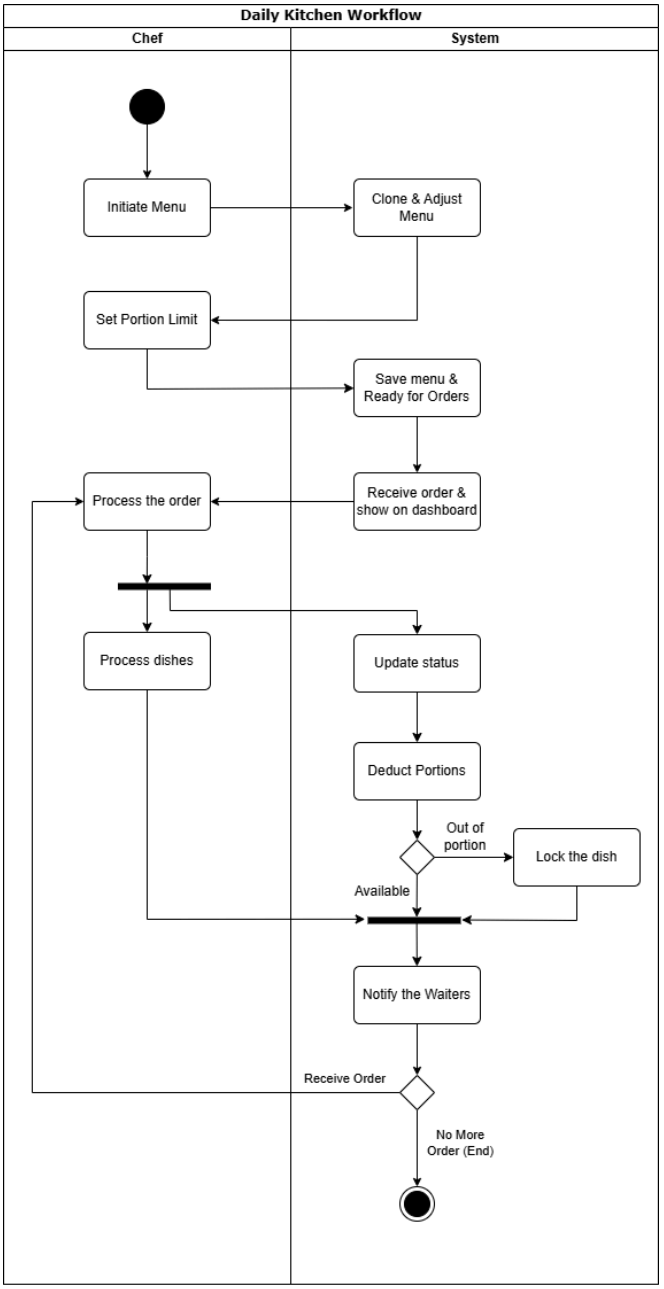
7.3.1 Waiter Order Processing



1. Select a serving table to either create a new table session or access an existing active table session.
2. Display the current menu, including dish name, price, short description, and real-time availability status.
3. Input customer selections, specify quantities, and attach optional notes for each dish.
4. Review and confirm the order, prompting the system to validate the input and check the portion availability of the selected dishes.
5. Evaluate availability through a decision node: if a dish reaches zero remaining portions, lock the dish immediately, prevent further selection, and display an out-of-stock message.
6. Generate a unique Order ID for the confirmed order.
7. Save the order data, and transmit the confirmed order to the kitchen order list.

8. Waiter receives customer's order, if they want to order more: back to step 3 - Input customer order, if they want to bill: continue.
9. Waiter initiates the billing process, the system calculates the total and generates the invoices.
10. Waiter confirms the payment completion and the system will close the table session.

7.3.2 Daily Kitchen Workflow

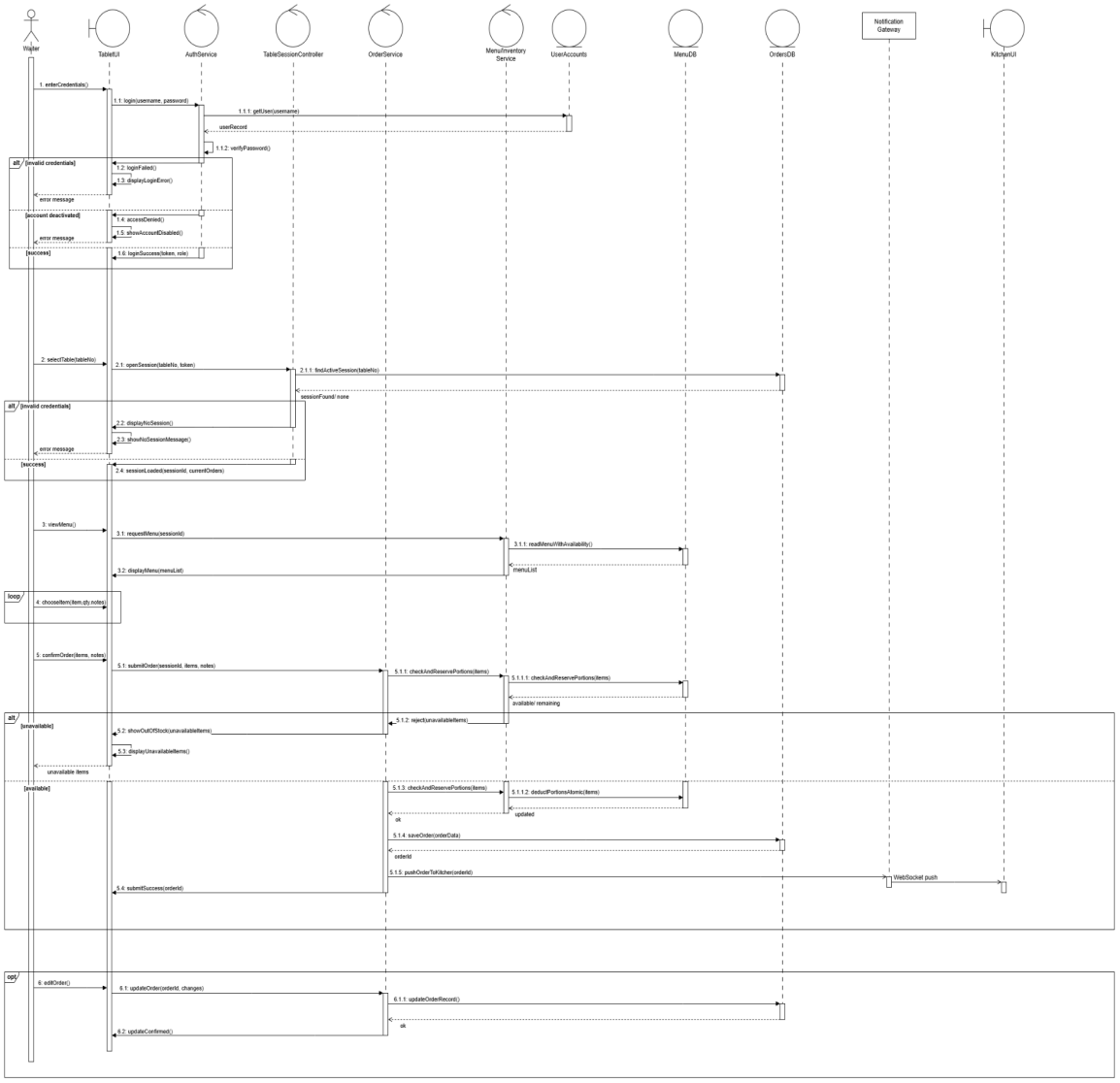


1. Begin the daily setup by copying the previous day's menu structure, including dishes, prices, descriptions, and notes, to the current day.
2. Set and save the specific daily portion limit for each dish.
3. Transition the system into an active monitoring state to wait for incoming orders.

4. Instantly push the details of a newly submitted order to the real-time kitchen dashboard, displaying the Order ID, items, quantities, and special notes.
5. Update the order status to "In Progress" as preparation begins, and subsequently to "Completed" when finished.
6. Trigger an immediate notification to the corresponding serving staff and update the table session upon each status change.
7. Continuously evaluate the remaining inventory throughout the execution cycle.
8. Trigger an automatic lock on an item via a decision node if its portion count drops to zero, instantly notifying all active tablets to prevent over-ordering.
9. Loop the workflow to await the next incoming order.

7.4 Sequence Diagrams

7.4.1 Waiter Order Processing

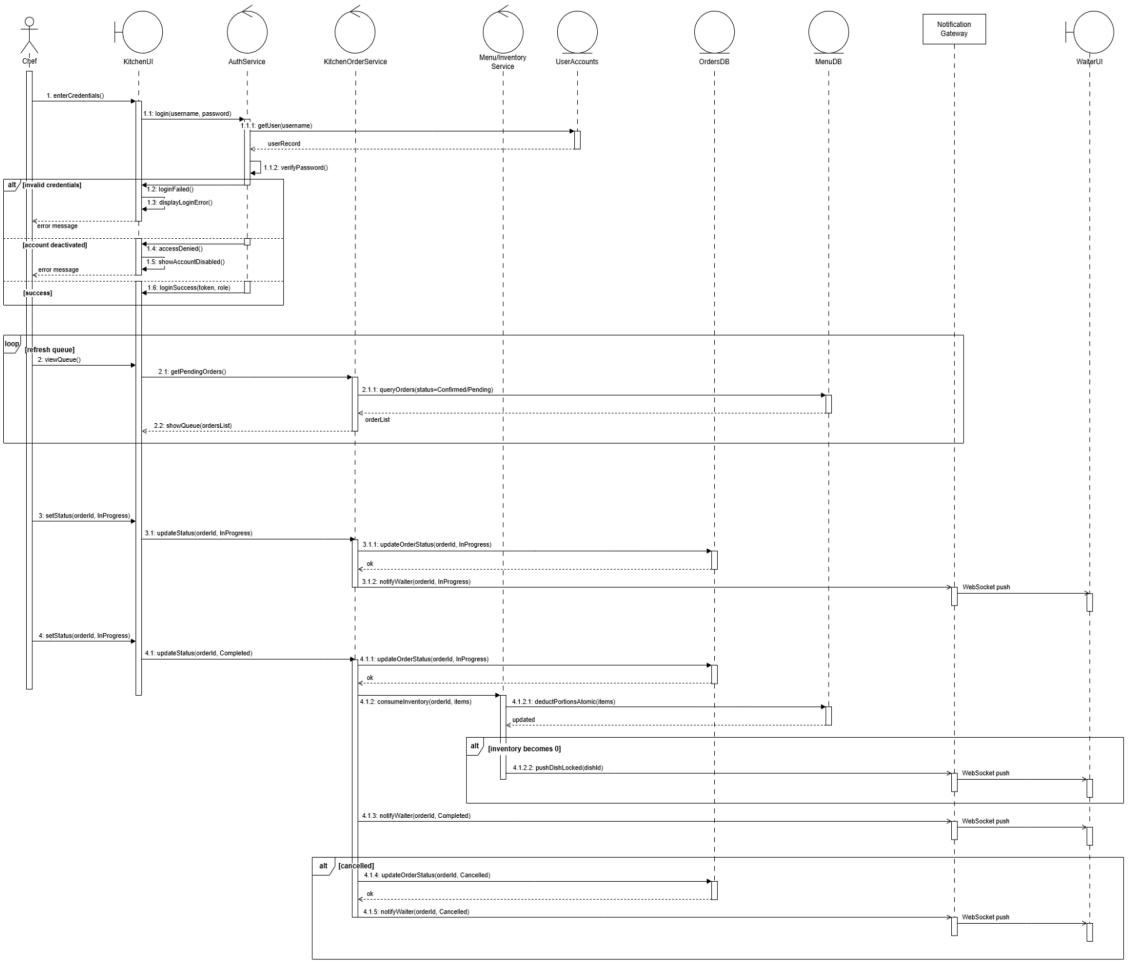


This sequence diagram describes the interaction between the Waiter and the Restaurant Management System during the digital ordering process. The workflow begins when the Waiter logs into the system through the Tablet UI, where the authentication request is processed by the Auth Service using the User Accounts database. After successful authentication, the waiter selects a table session and retrieves the menu associated with that session. The system then displays the available menu items, allowing the waiter to assist the customer in placing an order.

Once the order is confirmed, the Tablet UI sends the order details to the Order Service for processing. The Order Service coordinates with the Menu/Inventory Service to verify the availability of each requested item and reserve the necessary portions. If the items are available, the order is stored in the Orders database and a notification is sent to the kitchen through the Notification Gateway. The system then returns a confirmation message to the Tablet UI indicating that the order has been successfully submitted.

Several control frames are used to represent variations in the interaction flow. The alt frame models conditions such as invalid login credentials, deactivated accounts, or unavailable menu items. The loop frame represents the repeated selection of dishes before the order is finalized. In addition, the opt frame captures optional actions where the waiter may modify an existing order before submission. These frames help represent both normal and exceptional scenarios in the ordering process.

7.4.2 Daily Kitchen Workflow

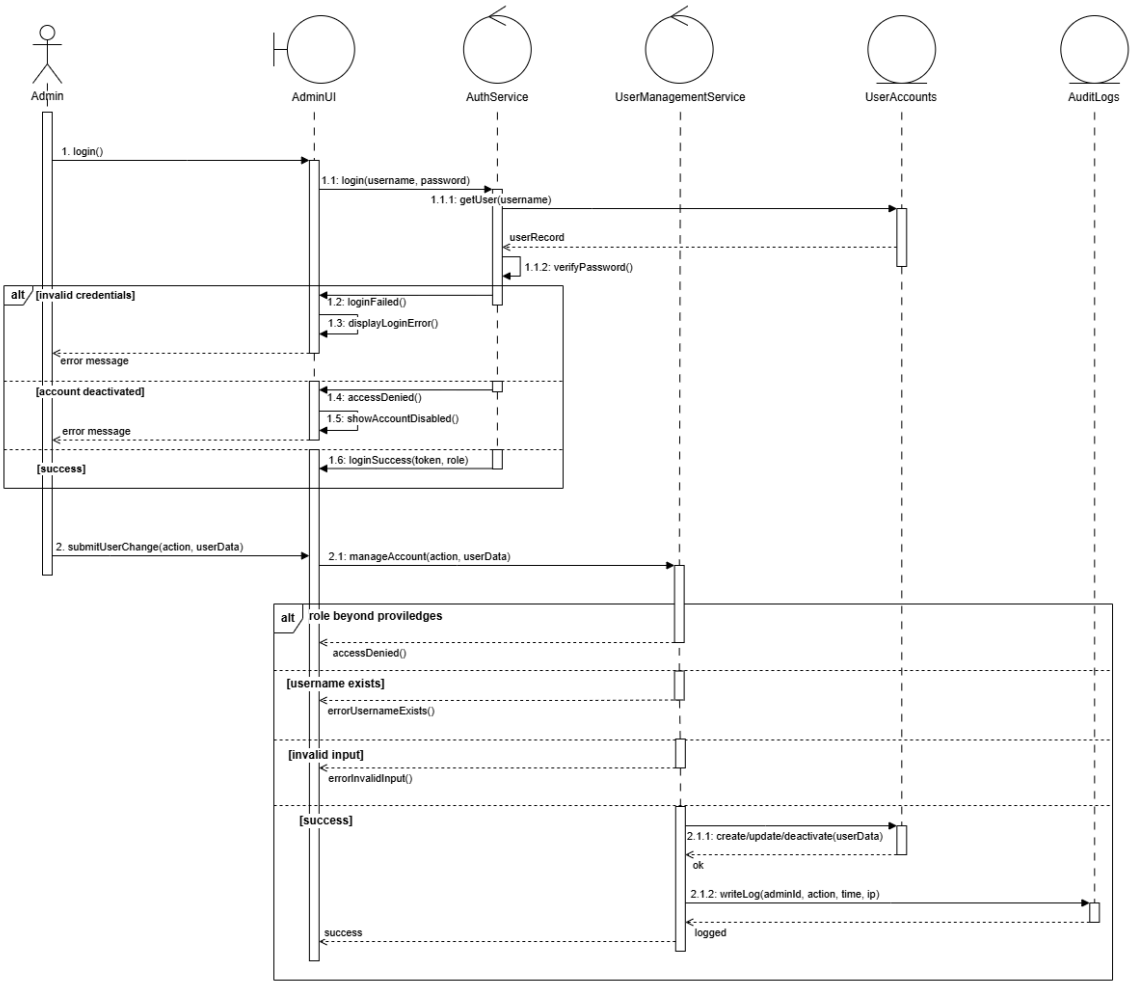


This sequence diagram illustrates how the chef interacts with the system to manage the kitchen order queue. The process begins with the chef logging into the Kitchen UI, where authentication is performed by the Auth Service using the User Accounts database. After successful login, the chef retrieves the list of pending orders through the Kitchen Order Service. The service queries the Orders database and returns the current order queue, which is then displayed on the kitchen interface.

When the chef begins preparing an order, its status is updated through the Kitchen UI. The Kitchen Order Service records the updated status in the Orders database and sends a notification to the waiter interface through the Notification Gateway. Once an order is completed, the system triggers an inventory update through the Inventory Service. The corresponding portions are deducted from the Menu database to ensure that inventory levels remain consistent with the completed order.

The diagram includes several control frames to represent alternative conditions and repeated actions. The loop frame models the repeated refreshing of the order queue as the chef monitors incoming orders during operation. The alt frame captures situations where an order is cancelled, requiring the system to update the order status and notify the waiter. Another alt branch represents the case where inventory for a menu item reaches zero, which triggers the system to lock that dish and send a notification to the relevant components.

7.4.3 System Administration

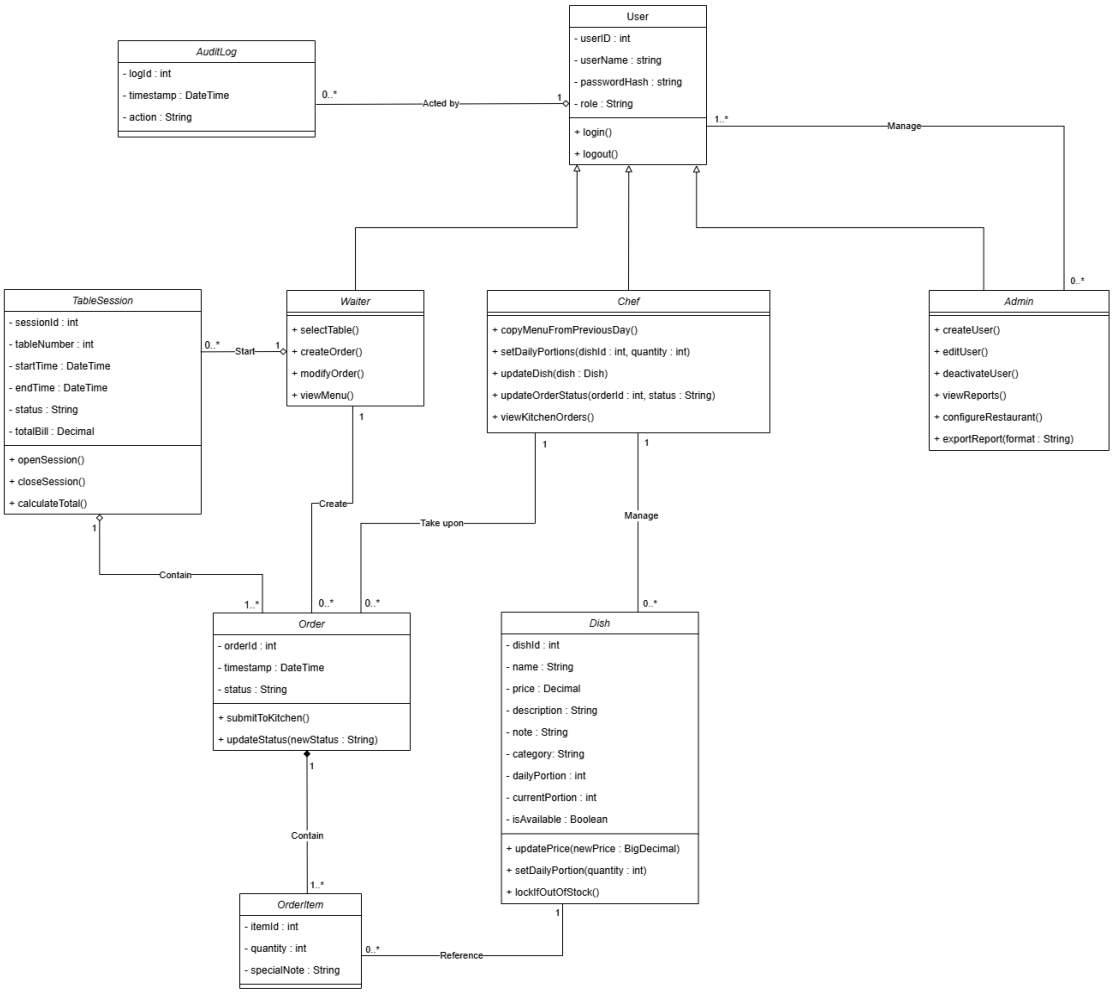


This sequence diagram describes how the administrator manages user accounts in the restaurant management system. The interaction begins when the administrator logs into the Admin UI, where the Auth Service verifies the credentials using the User Accounts database. After successful authentication, the administrator can submit requests to create, update, or deactivate user accounts.

The Admin UI forwards the request to the User Management Service, which processes the requested operation and updates the User Accounts database accordingly. Once the operation is completed, the system records the administrative action in the Audit Log to maintain a traceable history of account management activities. The result of the operation is then returned to the Admin UI as a confirmation message.

Control frames are used to represent validation and authorization conditions during account management. One alt branch handles cases where the submitted username already exists or when the input data is invalid, returning an appropriate error message to the administrator. Another alt branch represents situations where the administrator attempts to perform actions beyond their assigned privileges, resulting in an access denial response. These frames ensure that the system enforces both input validation and role-based access control during administrative operations.

7.5 Class Diagrams



The Class Diagram represents the static structure of the Restaurant Management System, illustrating the core classes and their relationships based on the system requirements. The diagram centers around the User class, which serves as the base class with inheritance relationships to the Waiter, Chef, and Admin subclasses to implement role-based access control (RBAC). Key domain classes include Dish (with dailyPortion, currentPortion, and lockIfOutOfStock() for real-time inventory management), TableSession (supporting session-based billing), Order and OrderItem (enabling live order modification), and AuditLog. A many-to-many association exists between Admin and User for management purposes, besides one-to-many relationships such as TableSession contains Order, Order contains OrderItem (which references Dish), Chef manages Dish, and AuditLog acted by User.

8. System Design

8.1 Architectural Design

8.1.1 Architecture and Design Pattern

The system adopts a client–server architecture combined with the Model–View–Controller (MVC) design pattern, implemented as a modular monolithic system. This architectural approach separates the system into a client layer, an application layer, and a data layer, while internally organizing the application logic using MVC principles.

The client–server architecture is chosen because the system is designed as a web-based application where multiple users, including waiters, chefs, and administrators, interact with a centralized system. This structure allows all business logic and data processing to be handled on the server side, while client devices are responsible only for presenting the user interface. As a result, the system ensures consistency, easier maintenance, and centralized control of data.

Within the application server, the MVC pattern is applied to separate concerns. The View layer handles user interfaces such as the Waiter UI, Chef UI, and Admin UI. The Controller layer processes user requests and coordinates system operations through modules such as Authentication, Order & Table, Kitchen, Menu & Inventory, and Admin. The Model layer manages data and business rules, including user data, order data, and menu information. This separation improves code maintainability, scalability, and clarity of system design.

8.1.2 Architectural Justification and Comparison

Several architectural approaches were considered for the system, including a traditional monolithic architecture, a microservices architecture, and the chosen client–server architecture combined with an MVC-based modular monolith.

A traditional monolithic architecture without clear layering offers simplicity in development and deployment, as all components are implemented within a single unified codebase. However, this approach tightly couples the user interface, business logic, and data handling. For this system, which includes multiple functional areas such as order processing, kitchen coordination, inventory management, and reporting, such tight coupling would reduce maintainability and make future extensions more difficult. Any modification in one part of the system could unintentionally affect other components, increasing development complexity over time.

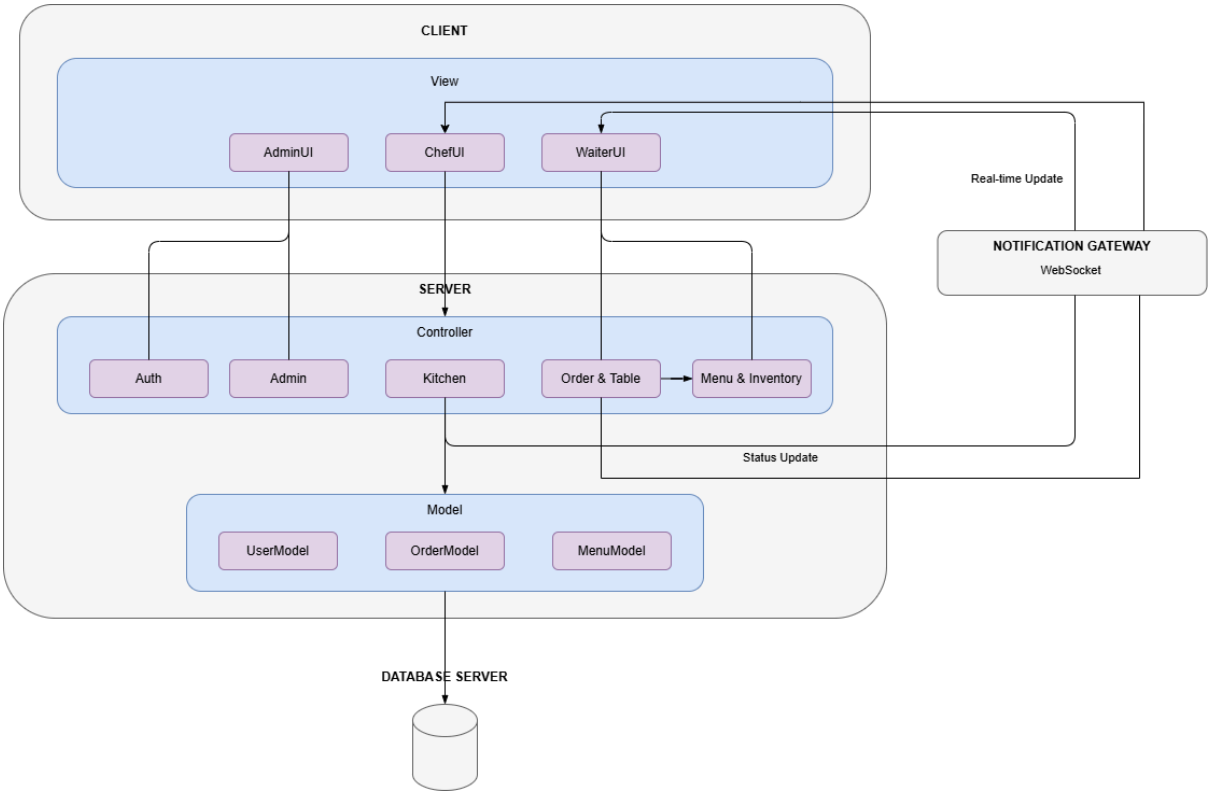
A microservices architecture was also considered due to its advantages in scalability and independent deployment. In this approach, each functional module would be deployed as a separate service, allowing flexible scaling and fault isolation. However, the system operates in a relatively

small to medium-scale environment with centralized usage within a single restaurant. As a result, the overhead of managing multiple services, handling inter-service communication, and maintaining distributed deployment would introduce unnecessary complexity. Additionally, real-time features such as order status updates would require more complex coordination across services, which is not justified for the scale of this system.

The chosen solution is a client–server architecture combined with an MVC-based modular monolith. This approach fits the system requirements effectively. The client–server model supports centralized data management and ensures consistency across different user roles, which is essential for operations such as order handling and inventory updates. Within the application server, the MVC pattern provides clear separation of concerns between the user interface, business logic, and data management, improving maintainability and extensibility. Furthermore, the modular structure within the controller layer allows different functional areas to remain logically independent while still being deployed within a single system.

In addition, the architecture supports the system's real-time requirements through the integration of a Notification Gateway using WebSocket, enabling efficient communication between the kitchen and waiter interfaces without introducing the complexity of distributed systems. Overall, this architecture achieves a balance between simplicity, performance, and modularity, making it the most suitable choice for the system.

8.1.3 Logical View



The logical architecture diagram illustrates the high-level structure of the system and the relationships between its main components. The system is divided into three primary layers: Client, Application Server, and Database Server.

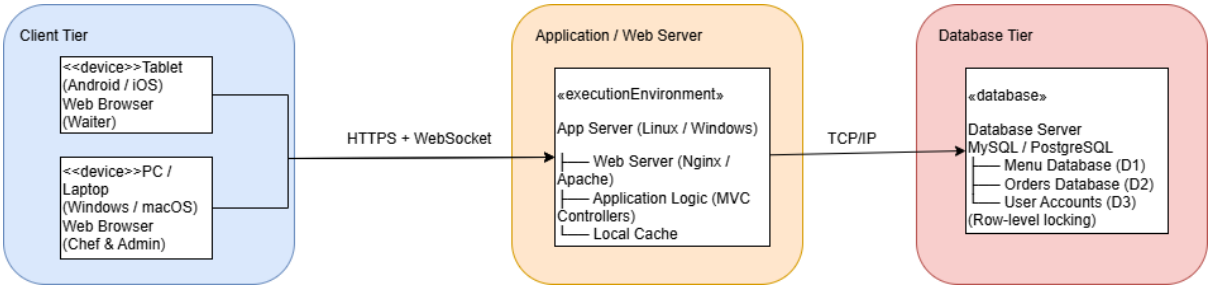
The Client layer represents the user interfaces used by different actors. The Waiter UI is responsible for order creation and menu interaction, the Chef UI handles order processing and status updates, and the Admin UI supports system management and reporting. These interfaces interact with the server through HTTP/HTTPS requests.

The Application Server contains the core logic of the system and follows the MVC structure. The Controller layer includes functional modules such as Authentication, Order & Table, Kitchen, Menu & Inventory, and Admin, each responsible for handling specific system operations. The Model layer manages data entities and enforces business rules, including user management, order processing, and inventory tracking.

The Database Server stores all persistent data, including user accounts, orders, and menu information. The Model layer interacts with the database to perform data retrieval and updates.

In addition, a Notification Gateway is included to support real-time communication. The Kitchen module sends status updates to the gateway, which then pushes real-time updates to the Waiter UI using WebSocket technology. This ensures that order status changes are immediately reflected without requiring manual refresh.

8.1.4 Physical View



The physical architecture diagram illustrates how the software components of the Restaurant Management System are deployed onto hardware nodes and how they communicate over the network using a multi-tier client/server architecture. The system is organized into three main tiers: Client, Application/Web Server, and Database Server.

The Client tier consists of two types of devices: Tablet Devices running Android or iOS with a standard Web Browser used by Waiters for order processing and menu interaction, and PC/Laptop Devices running Windows, macOS, or Linux with a Web Browser used by Chefs and Admins for kitchen operations and system administration. These clients connect to the Application Server through

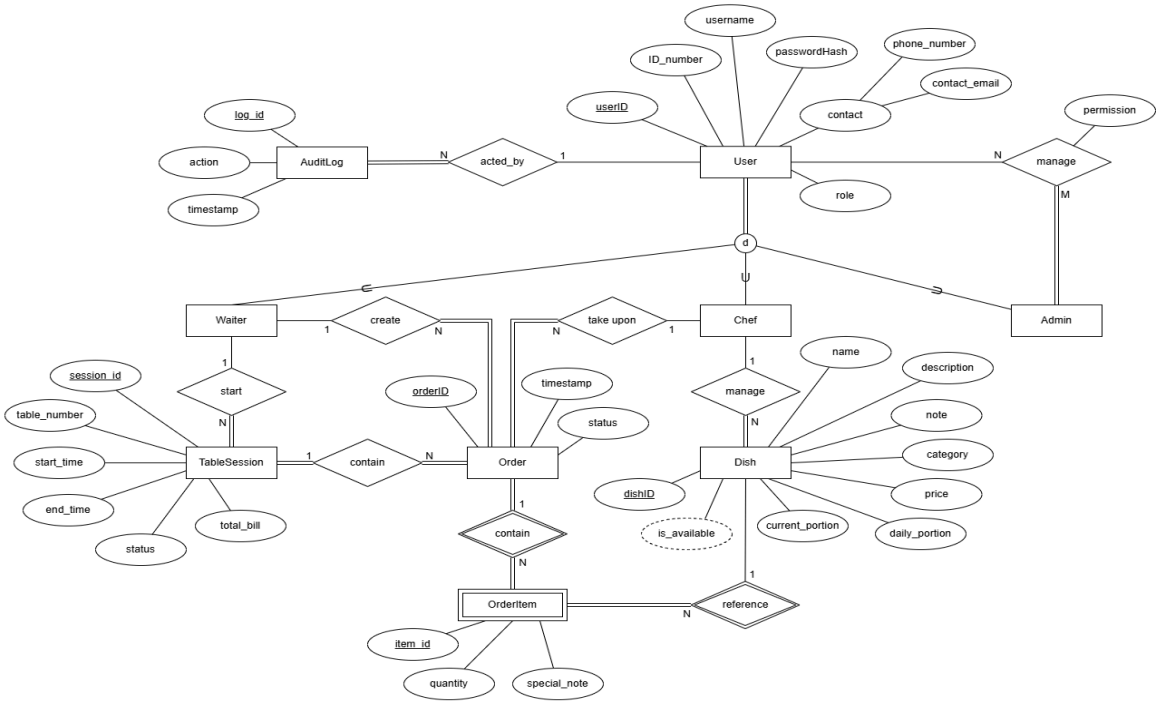
the internal Wi-Fi/LAN network using secure HTTPS and WebSocket protocols to support real-time updates.

The Application/Web Server runs on a dedicated Linux or Windows machine and hosts the complete application logic. It includes the Web Server layer for handling incoming requests, the Application Logic layer implementing the MVC controllers, and a Local Cache component to support temporary offline operations. This server acts as the central processing unit between clients and the database.

The Database Server, running MySQL or PostgreSQL, stores all persistent data including the Menu Database (D1), Orders Database (D2), and User Accounts Database (D3). It supports concurrent transactions with row-level locking to ensure data consistency when multiple users access the same dish simultaneously.

This deployment design ensures high performance, scalability, and reliability in a real restaurant environment while maintaining secure and real-time communication across all layers.

8.2 Database Design



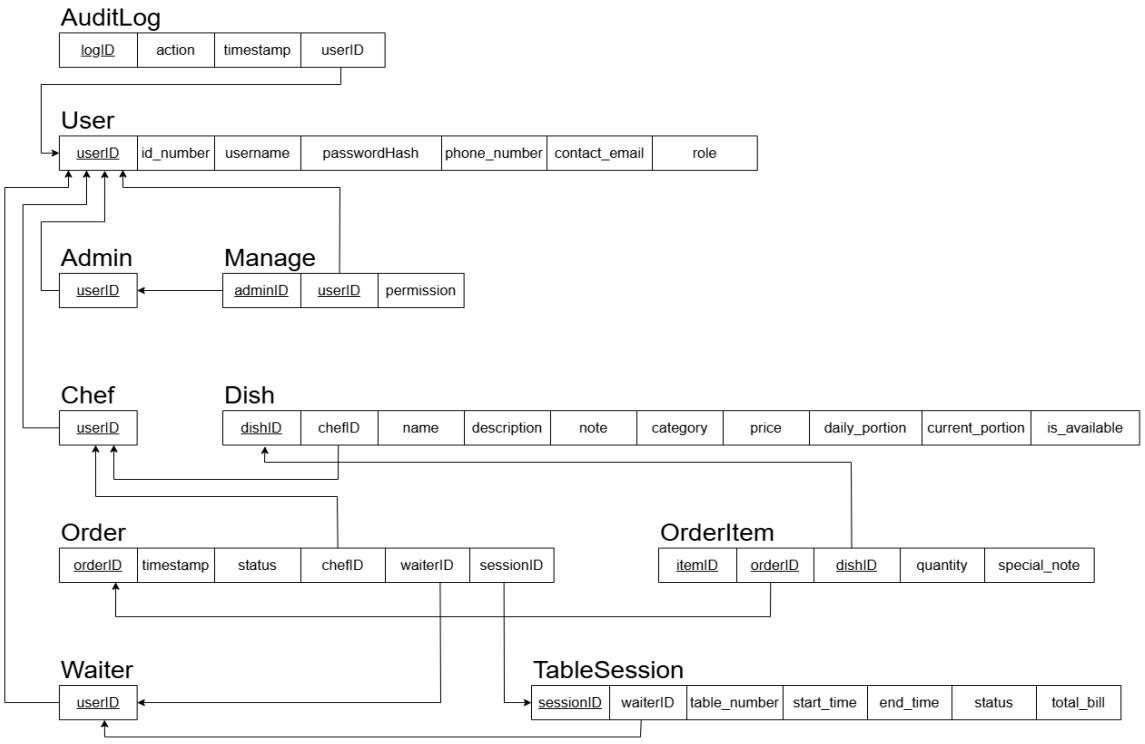
The RESTAURANT database keeps track of the restaurant's staff, menu offerings, dining sessions, and customer orders.

- The system manages all personnel under a ez general user profile. The database will store each user's unique ID, government ID number, username, hashed password, phone number,

and contact email. Users are categorized by their specific operational roles: Admin, Chef, and Waiter.

- To maintain system security and oversight, the database keeps a continuous audit log that records the specific action, timestamp, and the user who performed it. Additionally, Admins are responsible for managing other users in the system, and the database tracks the specific permissions granted to an Admin.
- The restaurant offers a variety of dishes. For each dish, the database tracks a unique ID, name, detailed description, menu category, and price. To manage kitchen inventory, the system records the maximum daily portion allowed, the current remaining portion for the day, its overall availability status, and any internal preparation notes. Chefs are directly responsible for managing these dishes.
- When customers dine in, a Waiter starts a table session. We keep track of the physical table number, the start and end times of the dining session, its status, and the accumulated total bill. A single Waiter is responsible for starting and overseeing this session.
- During a table session, customers can place multiple orders. A Waiter creates the order, and a single Chef takes it upon themselves to prepare it. The database tracks the order's timestamp and current preparation status.
- Each order contains specific order items. For every individual item requested, the database records the specific dish being ordered, the quantity desired, and any special note made by the customer.

Relations after mapping the ERD/EERD to the relational data model:



User

Attribute	Data Type	Key	Description
userID	INT	PK	Unique identifier for the user.
id_number	VARCHAR		Government or internal identification number.
username	VARCHAR		Login username for system access.
passwordHash	VARCHAR		Encrypted password for authentication.
phone_number	VARCHAR		User's contact phone number.
contact_email	VARCHAR		User's contact email address.
role	VARCHAR		Defines the base role (e.g., Admin, Chef, Waiter).

Admin

Attribute	Data Type	Key	Description
userID	INT	PK, FK	References User(userID).

Chef

Attribute	Data Type	Key	Description
userID	INT	PK, FK	References User(userID).

Waiter

Attribute	Data Type	Key	Description
userID	INT	PK, FK	References User(userID).

AuditLog

Attribute	Data Type	Key	Description
logID	INT	PK	Unique identifier for the audit log entry.
action	VARCHAR		Description of the action performed in the system.
timestamp	DATETIME		Date and time the action occurred.
userID	INT	FK	References User(userID) who performed the action.

Manage

Attribute	Data Type	Key	Description
adminID	INT	PK, FK	References Admin(userID).
userID	INT	PK, FK	References User(userID) being managed.

permission	VARCHAR		Specific permissions granted by the Admin to the User.

TableSession

Attribute	Data Type	Key	Description
sessionID	INT	PK	Unique identifier for the dining session.
waiterID	INT	FK	References Waiter(userID) managing the table.
table_number	INT		The physical table number in the restaurant.
start_time	DATETIME		When the table session was opened.
end_time	DATETIME		When the table session was closed.
status	VARCHAR		Current session status (e.g., Active, Paid, Closed).
total_bill	DECIMAL		Total accumulated cost for the session.

Dish

Attribute	Data Type	Key	Description
dishID	INT	PK	Unique identifier for the menu item.
chefID	INT	FK	References Chef(userID) who manages this dish.
name	VARCHAR		Name of the dish.
description	TEXT		Detailed description or ingredients of the dish.
note	VARCHAR		Internal notes (e.g., allergy warnings, prep time).
category	VARCHAR		Menu category (e.g., Starter, Main Course, Dessert, Drinks).
price	DECIMAL		Selling price of the dish.
daily_portion	INT		Maximum number of portions available to sell per day.
current_portion	INT		Remaining portions available for the current day.
is_available	BOOLEAN		Derived status indicating if the dish can currently be ordered.

Order

Attribute	Data Type	Key	Description
orderID	INT	PK	Unique identifier for the specific order/ticket.
timestamp	DATETIME		Date and time the order was placed.
status	VARCHAR		Current state of the order (e.g., Pending, Preparing, Served).
chefID	INT	FK	References Chef(userID) assigned to fulfill the order.
waiterID	INT	FK	References Waiter(userID) who placed the order.
sessionID	INT	FK	References TableSession(sessionID) this order belongs to.

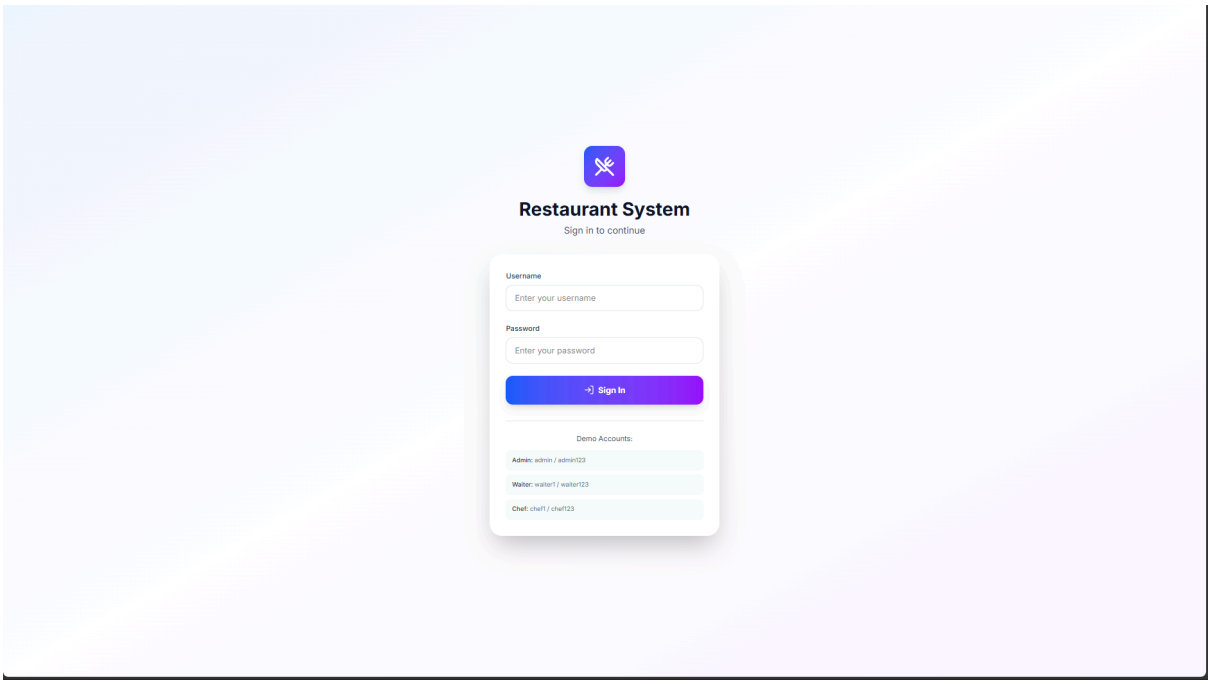
OrderItem

Attribute	Data Type	Key	Description
itemID	INT	PK	Unique identifier for the individual line item.
orderID	INT	FK	References Order(orderID) this item belongs to.
dishID	INT	FK	References Dish(dishID) being ordered.
quantity	INT		Number of servings requested for this specific dish.
special_note	VARCHAR		Specific customer requests (e.g., "no onions", "sauce on side").

8.3 UI Design

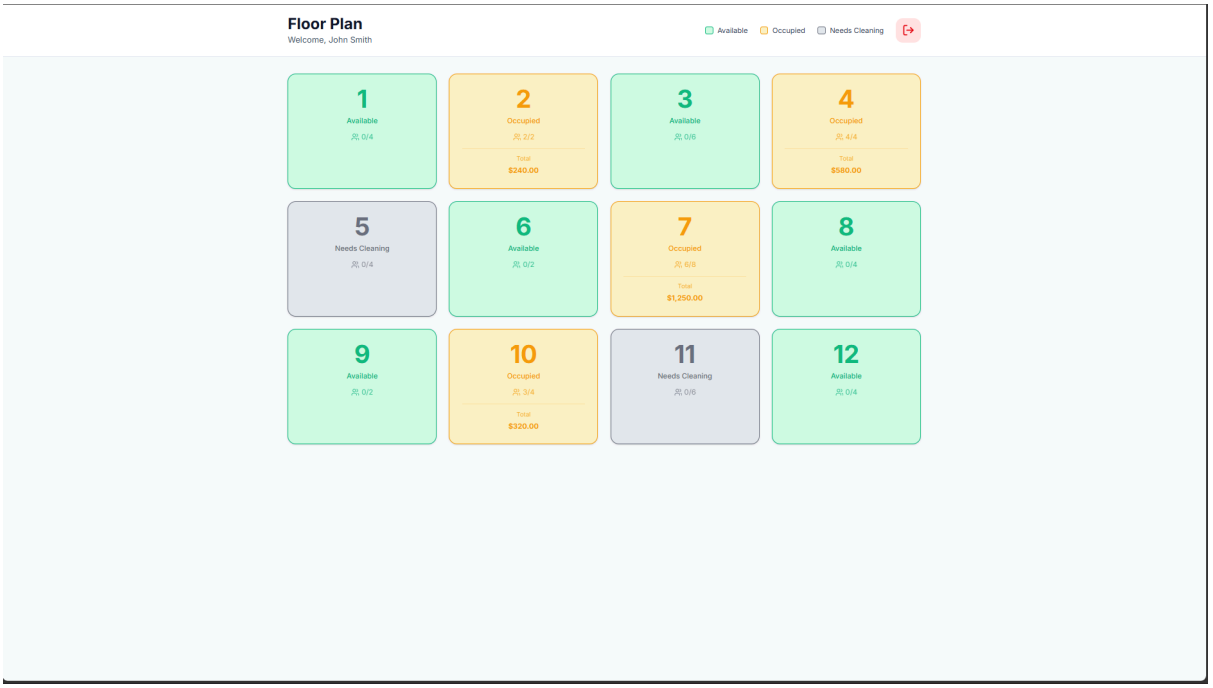
The user interface (UI) of the Restaurant Management System is designed with a "tablet-first" approach, ensuring that touch targets are large, navigation is intuitive, and information is highly visible in fast-paced restaurant environments. The design employs a clean, modern aesthetic with consistent color-coding to convey status instantly.

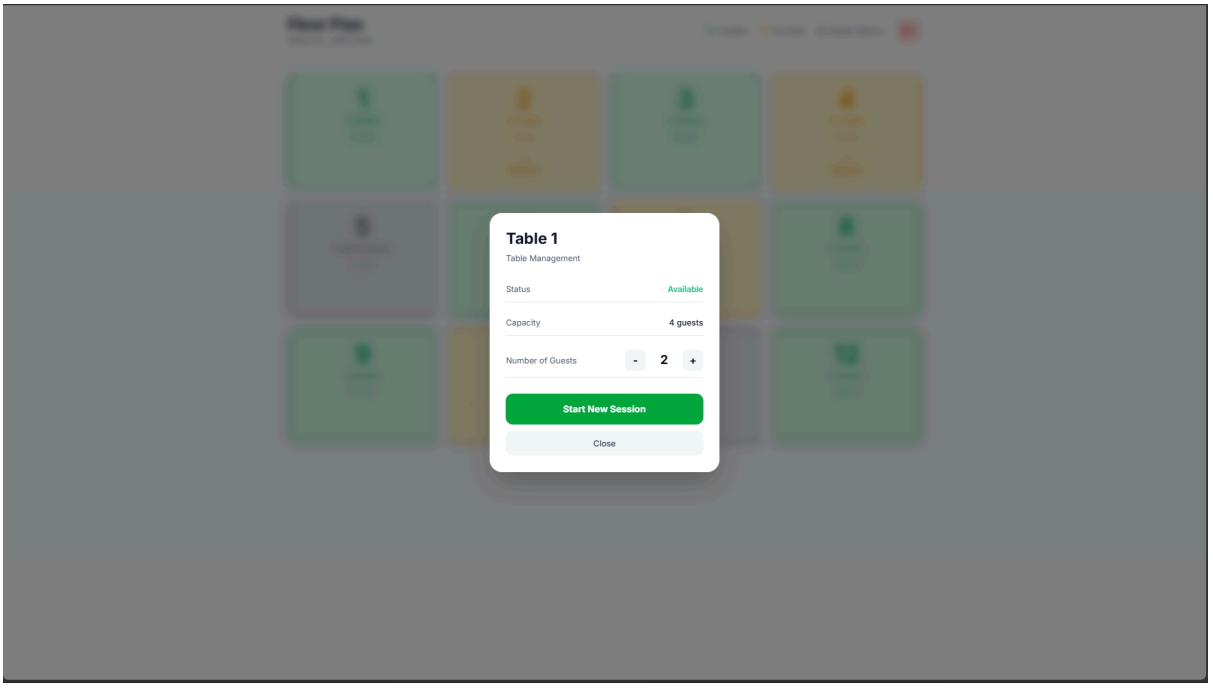
8.3.1 Authentication Interface



Login Screen: Serves as the centralized secure gateway for all users. It features a clean, minimalist card-based layout centered on the screen, containing input fields for the Username and Password, and a prominent "Sign In" button. This ensures a distraction-free entry point for staff.

8.3.2 Waiter Interface (Order Processing & Table Management)





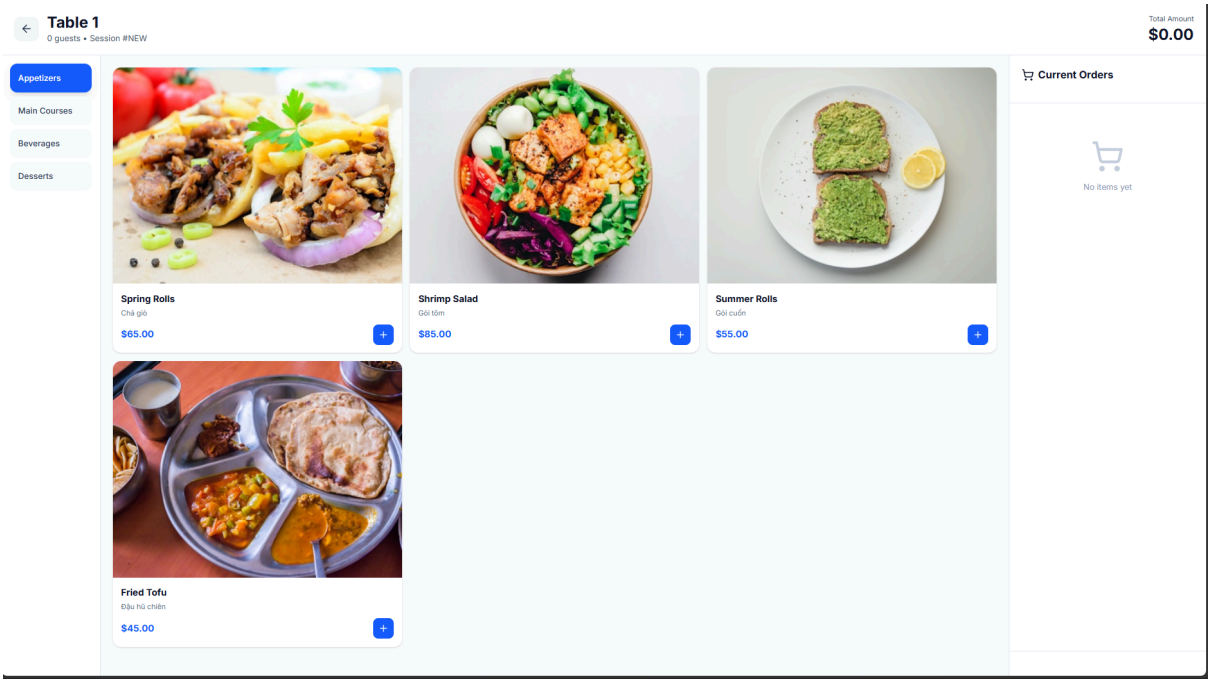


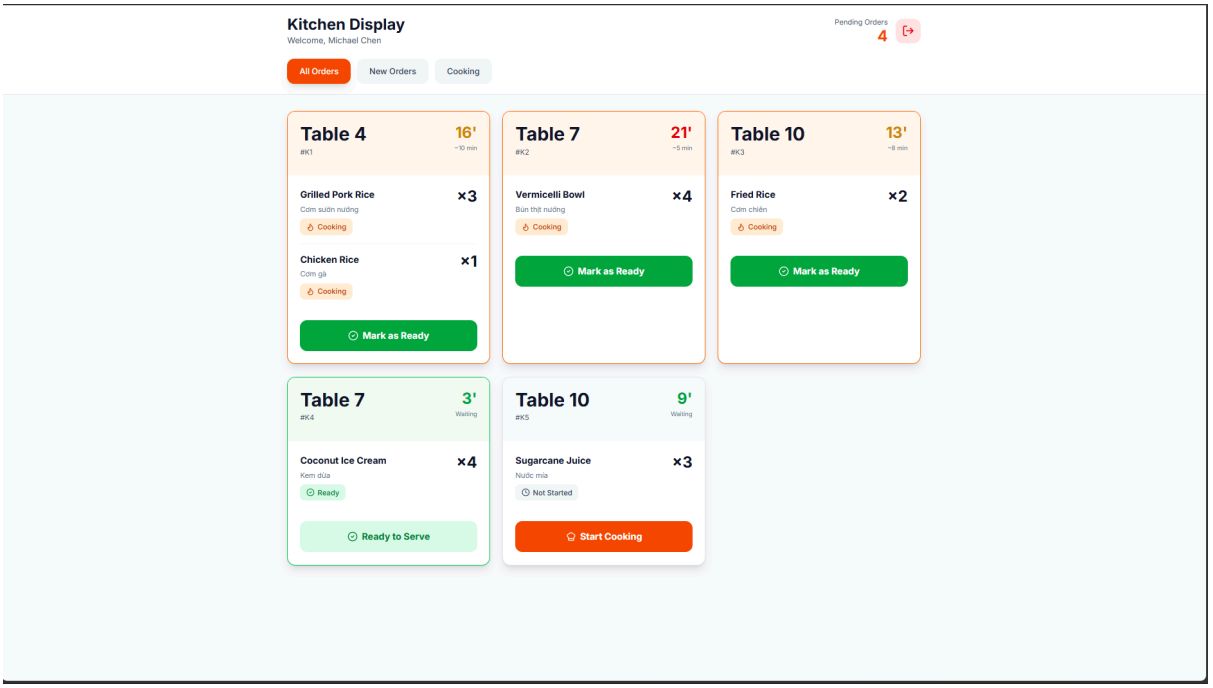
Table Floor Plan: This is the primary dashboard for Waiters. It displays a visual grid of all restaurant tables. The system utilizes intuitive color-coding for rapid status recognition: Green for "Available," Yellow for "Occupied," and Gray for "Needs Cleaning." Each table card displays its current capacity and the total bill accumulated so far.

Session Initiation Modal: When a Waiter taps an available table, a clean modal overlay appears, allowing the staff to input the specific number of guests before tapping "Start New Session," seamlessly initiating the billing and ordering process.

Make Order Screen: Designed for rapid item entry, this screen uses an efficient three-column layout:

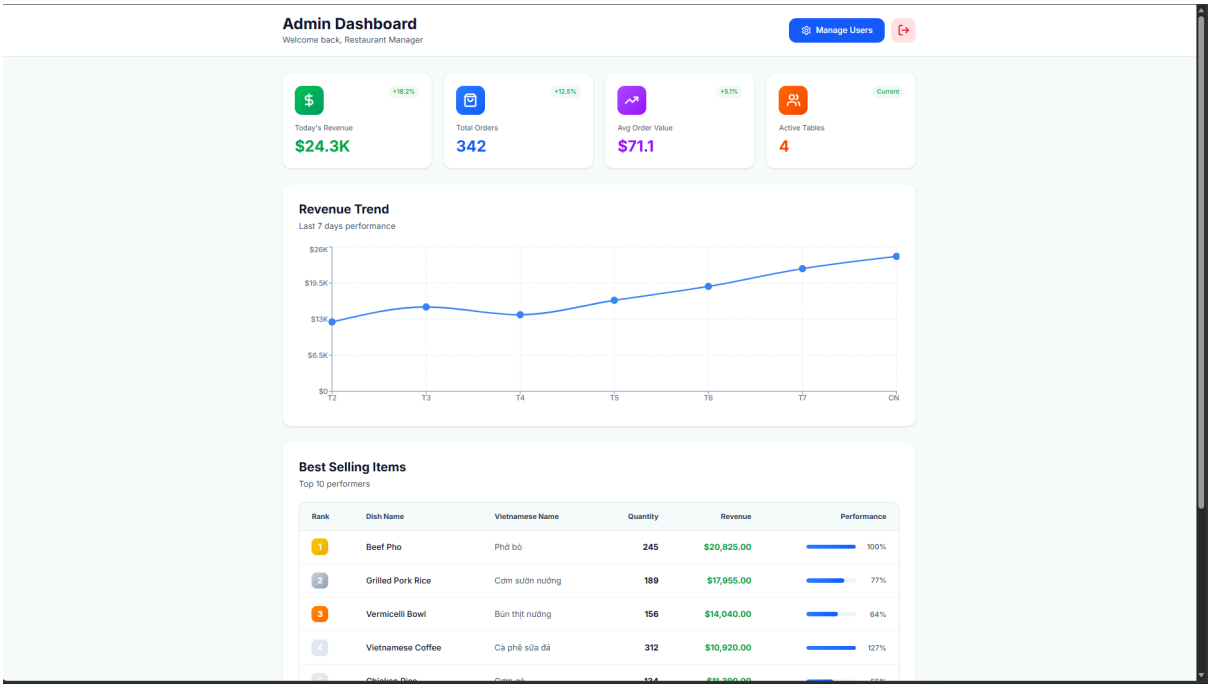
- **Left Pane:** A vertical navigation menu for dish categories (e.g., Appetizers, Main Courses, Beverages).
- **Center Pane:** A visually rich grid displaying dish cards. Each card includes a high-quality image, dish name, price, and a quick-add "+" button.
- **Right Pane:** The "Current Orders" cart, providing a real-time summary of selected items and the total amount before submission.

8.3.3 Chef Interface (Kitchen Operations)



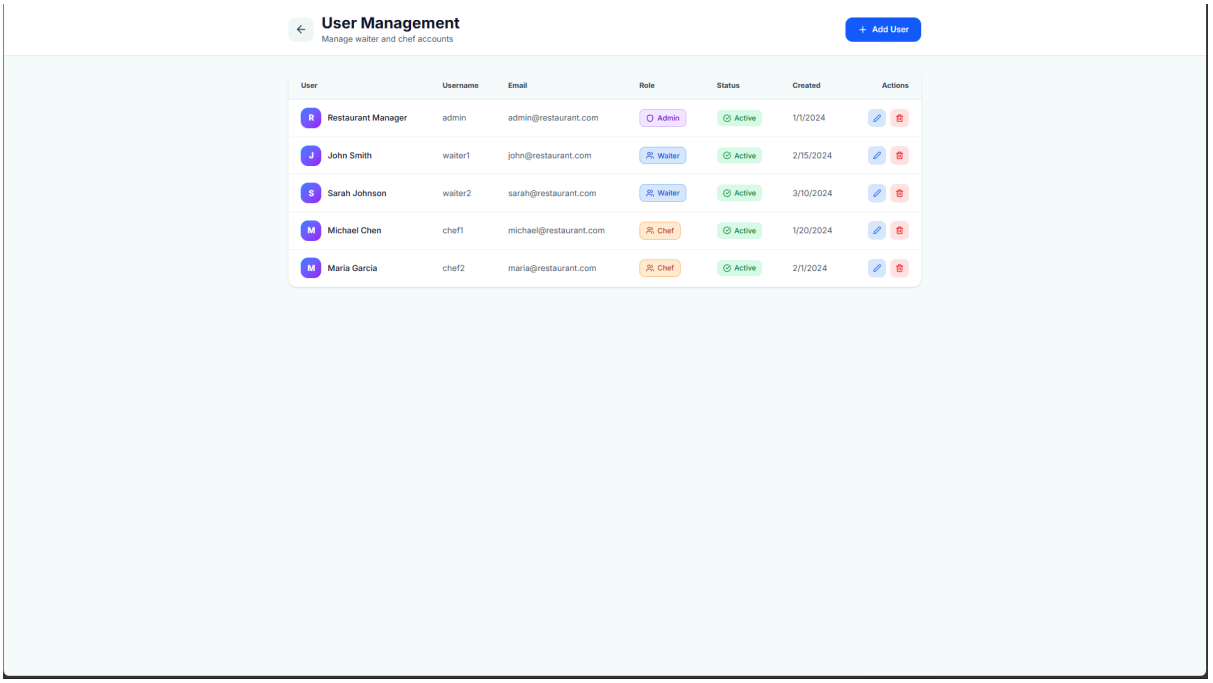
- **Kitchen Display System (KDS):** The kitchen interface abandons traditional lists in favor of a highly readable, card-based ticket system.
    - **Order Tickets:** Each card represents an active table order, displaying the wait time (timer in minutes), dish names, and required quantities.
    - **Status Indicators & Actions:** Dishes are tagged with their current preparation status (e.g., "Not Started," "Cooking," "Ready"). Large, brightly colored action buttons (like "Start Cooking" or "Mark as Ready") allow Chefs to update the system with a single tap.
    - **Navigation:** Top tabs allow the kitchen staff to quickly filter views between "All Orders," "New Orders," and "Cooking."

8.3.4 Admin Interface (System Administration & Reporting)

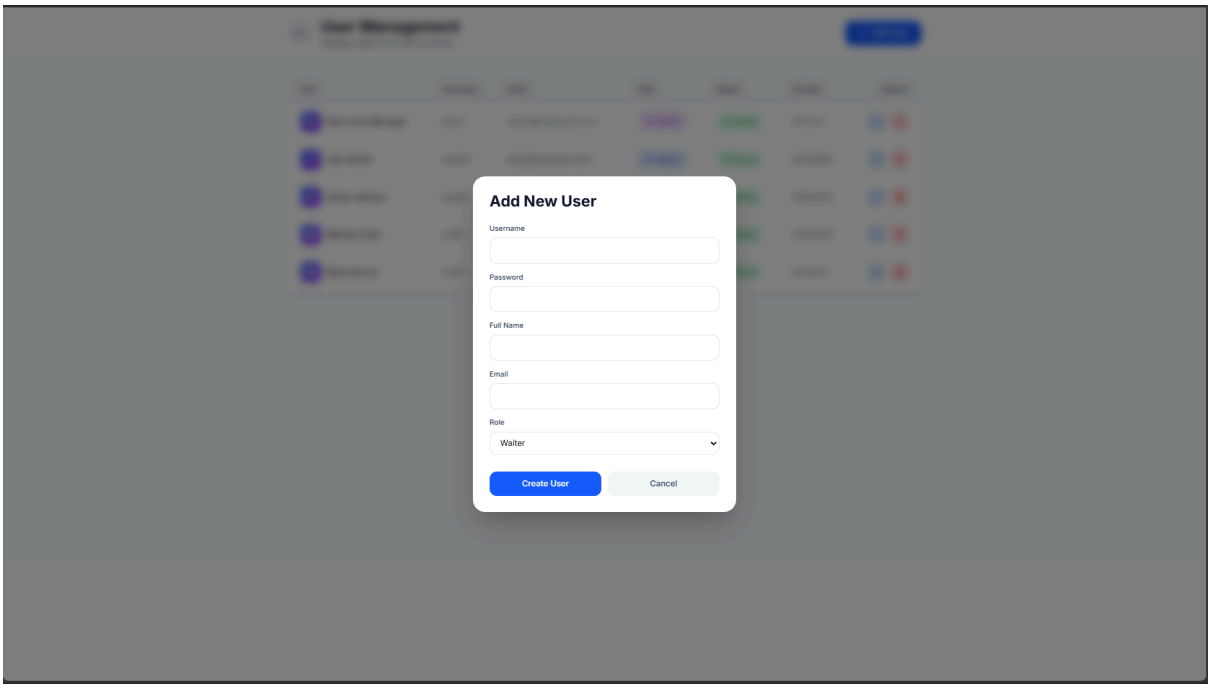


Admin Dashboard: Provides the restaurant manager with immediate insights into business performance.

- **KPI Cards:** The top section features distinct cards showing real-time metrics such as Today's Revenue, Total Orders, Average Order Value, and currently Active Tables.
- **Data Visualization:** A prominent "Revenue Trend" line chart tracks performance over the last 7 days.
- **Rankings:** A "Best Selling Items" table lists top-performing dishes with exact quantities and revenue data.



User Management Screen: A clean, tabular interface that lists all active personnel (Waiters, Chefs, Admins). It displays usernames, emails, assigned roles (color-coded for quick scanning), and account statuses. Action buttons allow for quick editing or deletion of accounts.



Add New User Modal: A straightforward form overlay that allows the Admin to input details for new hires, including full name, credentials, and a dropdown menu to explicitly assign their system role.

9. Conclusions

This Software Requirements Specification defines the scope and requirements for the Restaurant Management System, a mobile web application intended for a single-unit restaurant. Its primary goal is to digitize and automate the food ordering workflow, resolving key issues like manual error, inventory discrepancies, and complicated billing. The document details the system's core features, including digital order creation, real-time dynamic inventory control, and session-based billing, along with the distinct roles and responsibilities of the Waiter, Chef, and Admin users. Overall, the Software Requirements Specification serves as the definitive reference for stakeholders, providing the foundation for the design and implementation of an efficient, accurate, and secure restaurant operation.